



A model-driven formal methods approach to software architectural security vulnerabilities specification and verification[☆]

Quentin Rouland^{a,*}, Brahim Hamid^b, Jason Jaskolka^c

^a Computer Science and Engineering Department, Université du Québec en Outaouais, 283 Alexandre-Taché Boulevard, Gatineau, J8X 3X7, QC, Canada

^b IRIT, Université de Toulouse, 118 Route de Narbonne, Toulouse, 31062, Occitanie, France

^c Systems and Computer Engineering, Carleton University, 1125 Colonel By Drive, Ottawa, K1S 5B6, ON, Canada

ARTICLE INFO

Dataset link: <https://gitlab.com/semcoproject/toc>

Keywords:

Engineering secure systems
Model-driven approach
Software architecture
Vulnerability
Formal methods
Metamodel

ABSTRACT

Detecting and addressing security vulnerabilities in software designs is crucial for ensuring the reliable and safe operation of systems. Existing approaches for vulnerability specification lack the necessary flexibility for practical use. To tackle this issue, we propose an integrated model-driven approach for vulnerability detection and treatment during software architecture design. The approach involves specifying vulnerabilities as properties of a modeled system in a technology-independent language, expressing conditions for vulnerability detection using a language supported by automated tools, and recommending security requirements to mitigate detected vulnerabilities. Formalized vulnerabilities and security requirements are presented as model libraries to facilitate reuse. Our methodology employs first-order and modal logic as a technology-independent formalism, with Alloy as the tool-supported language for modeling and software development. We have developed a Model-Driven Engineering (MDE) tool to implement this approach. To validate our work, we apply it to representative vulnerabilities based on the Common Weakness Enumeration (CWE) classifications within the context of secure component-based software architecture development.

1. Introduction

The shift from traditional computer systems towards the Internet of Things, cloud-based applications, wireless communications, and other interfaces, requires a reconsideration of secure and trusted system engineering processes. Moreover, security is a quality attribute that is becoming increasingly critical in current software-intensive systems (Anderson, 2020) and is thus gaining substantial attention by the research and industrial community (Barnabe et al., 2011).

From the system developer's perspective, security issues are not only detected in code. They need to be identified early in the first development steps and at the highest levels, primarily at the architecture design stage, where their semantics are understood. They also need to be mapped to lower levels, where they are enforced by corresponding concrete mechanisms. For instance, the Microsoft Security Development Lifecycle (SDL) (Microsoft, 2024) addresses some of these issues and provides support for performing threat and abuse case modeling during the design stage.

In the context of software architecture, vulnerabilities denote inherent flaws in the design of a system that may be exploited to compromise its security. Detection of vulnerabilities in the design phase is very

useful when it comes to identifying weaknesses in early stages of system development. Identified risks can then be re-evaluated and be treated accordingly. While software architecture is usually expensive to change once system development is done, it is well-known nowadays that early detection and treatment of weaknesses during system development cycle allows one to save cost and time. However, the complexity of secure systems engineering requires automated tool support where the precise definition and specification of a vulnerability become a prerequisite.

Therefore, as a contribution to these efforts, we propose a model-driven approach to software architectural security vulnerabilities specification and verification. This approach employs formal methods to precisely specify and analyze security architecture vulnerabilities as properties of a modeled system. From an informal description of vulnerabilities from standards and classifications such as Common Weakness Enumeration (CWE) (MITRE Corporation, 2024b) or Common Attack Pattern Enumerations and Classifications (CAPEC) (MITRE Corporation, 2024a) in the context of component-based software architecture development, a logical specification of these properties is proposed using an abstract system computing model with a technology-independent

[☆] Editor: Raffaella Mirandola.

* Corresponding author.

E-mail addresses: quentin.rouland@uqo.ca (Q. Rouland), brahim.hamid@irit.fr (B. Hamid), jason.jaskolka@carleton.ca (J. Jaskolka).

specification language such as first-order logic, modal logic, Z (Spivey, 1989), Linear Temporal Logic (LTL) (Pnueli, 1977) or Computation Tree Logic (CTL) (Clarke and Emerson, 1982). Then, a concrete specification of the system computing model is done with a language with automated tool support such as Alloy (Jackson, 2006), nuSMV (Cimatti et al., 1999), SPIN (Holzmann, 2003) or UPPAAL (Larsen et al., 1997). Finally, a set of security requirements are elicited as properties of a model system to constrain the operations of the system and to treat the identified vulnerabilities. To evaluate our approach, we explore a set of representative vulnerabilities from categories based on the CWE vulnerability classification, targeting the components and the communication links in component-port-connector architecture views (Crnkovic, 2005).

The remainder of the paper is organized as follows. In Section 2, we describe the rationale behind selecting Alloy as the formal language used in this work and provide preliminaries on its use for our proposed approach. Section 3 introduces an illustrative example that will be used throughout the paper to explain the main concepts and give a general overview of our approach. Section 4 explains how we formally describe vulnerabilities using first-order and modal logic, along with categories from the CWE catalog. Section 5 outlines our method for formalizing and verifying these vulnerabilities using Alloy. Section 6 shows how we apply our approach to a specific illustrative example. Section 7 describes a Model-Driven Engineering (MDE) tool to support the proposed approach. Section 8 discusses the generalization and limitations of this work and how it can be integrated into the Software Development Life Cycles (SDLC). Section 9 explores related research with this work. Section 10 concludes and suggests areas for future research.

2. Preliminaries on Alloy

In this section, we present the rationale for choosing Alloy as the formal language and provide the necessary preliminaries on Alloy to illustrate our proposed approach.

2.1. A suitable formal language: Alloy

While any suitable formal language with tool support such as nuSMV, SPIN, UPPAAL, etc., can be used in our proposed approach for modeling and verification, in the following we choose Alloy because of the simplicity and straightforwardness of its analyzer. The Alloy Analyzer (Software Design Group (MIT), 2024) supports visualizing models and verifying their static and dynamic properties (e.g., behavioral aspects). It uses a constraint solver providing simulation and checking to find model instances satisfying the constraints defined during the model specification process. This makes it an appropriate candidate for our intended research work, i.e., specifying reusable security vulnerabilities and requirement libraries for software architecture design. In our work, the Alloy Analyzer essentially acts as a model checker and counterexample generator.

An advantage of Alloy's modeling approach compared to traditional model checking tools is its capability to handle partial models. With Alloy, architects can specify only a subset of constraints that the resulting system should satisfy. This enables us to construct models incrementally, allowing rapid iterations between modeling and analysis when writing a specification. A recent article (Jackson, 2019) has further highlighted the strengths of Alloy for software design. These strengths provide additional motivation and justification for adopting Alloy in this work.

However, scalability remains a concern when working with Alloy on larger architectures. To address this, additional techniques should be explored, such as integrating a decomposition of architectural models by breaking down the model into manageable components, as proposed in Baak et al. (2023). This approach enables the handling of larger-scale systems.

2.2. An introduction to Alloy

Alloy is a lightweight formal modeling language based on the first-order relational logic (Jackson, 2006). It was deeply inspired by Z (Spivey, 1989) and influenced by different object-oriented modeling languages such as UML. An Alloy model is composed of a set of *signatures* each defining a set of *atoms*. Atoms may have *fields* which define relations between atoms. In addition, signatures serve as types, and sub-typing may be defined as signature extension. Structure and behavior of the model are specified using constraints that correct instances of the model satisfy. There are several ways to specify constraints in the model. One is to treat them as *facts* that should hold at all times. Another one is to treat them as *predicates* defined in the form of parameterized formulas that can be used elsewhere and as *assertions* that are intended to follow from the facts of a model. In some situations, *functions* in the form of parameterized expressions may be used as helpers in the specification and verification processes.

The architect can instruct the Alloy Analyzer to verify whether the property of the system design holds, which would exhaustively explore every model instance within a scope of n . If it does not hold, a counterexample will be generated which can be visualized. The absence of counterexamples guarantees that the property holds in the modeled system, within the specified scope. As claimed in Jackson (2006), most counterexamples are found in a reasonably small scope. In our work, we set 5 as the experimental scope, defining it as the upper limit of the number of atoms within the sets of signatures. For example, this means that Alloy could instantiate a maximum of five interactions in a model using up to five components (attackers). While we believe this scope adequately addresses the objectives outlined in the paper, it is important to acknowledge that more sophisticated vulnerabilities may require us to expand the scope further. In the context of component-based software architecture, we studied the specification of the representative set of vulnerabilities and their verification in a reasonable time.

For the purpose of this paper, we formalize software architecture models and their semantics following some properties (specifications are presented in Section 4) to identify the presence of vulnerabilities and to provide a set of requirements to treat them. We facilitate this by providing an architectural metamodel in Alloy and an Alloy implementation of the vulnerabilities and requirements in the form of reusable libraries with respect to the semantics presented in Section 4. *A complete presentation of our Alloy metamodel, the full specification of the other constructs (e.g., communication primitives, etc.) and the reusable libraries are available online via <https://gitlab.com/semcoprocject/toc>.*

3. Approach

The approach presented in this paper supports the identification and validation of the treatments of software architecture vulnerabilities. The goal is to formalize the security requirements of the system architecture determined through vulnerability modeling. It enables the identification of vulnerabilities (i.e., software architecture security flaws) of a system architecture by means of verification of desired properties over a model of the architecture. It also enables the suggestion of new security requirements to improve the system design and to treat the identified security violations. The following outlines the overall development process, commencing with an illustrative example designed to contextualize the proposed methodology.

3.1. An illustrative example

We propose the following example to illustrate some vulnerabilities which can be found in system architectures, and how our approach can help to detect and counter them. The example is based on an Open Web Application Security Project (OWASP) example of a college library web application (OWASP, 2024). The UML class diagram in Fig. 1 shows the overall architecture of the web application, where software components

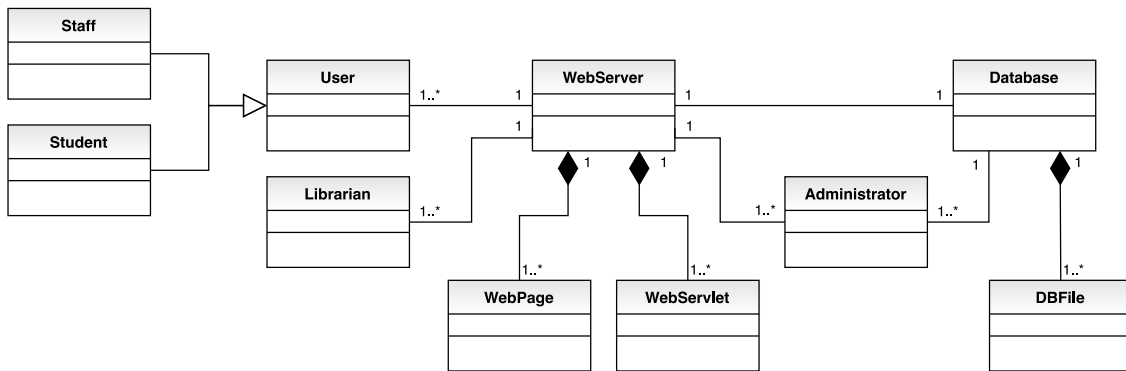


Fig. 1. Class diagram of a college library web application.

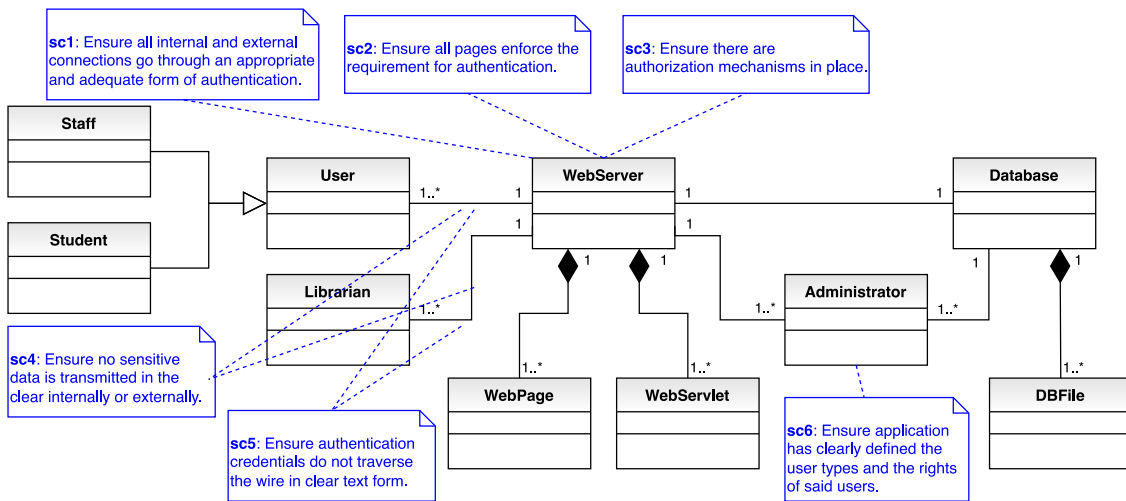


Fig. 2. College library web application example with security controls.

are represented by classes, and their relationships are represented by associations. It consists of several software components including a client (user), a web server, and a database.

The website provides online services to search for and to request books. The users are students, college staff, and librarians. Staff and students will be able to log in and search for books, and staff members can request books. Librarians will be able to log in, add books, add users, and search for books.

Fig. 2 shows the class diagram with a set of security controls (depicted in UML comment boxes) from OWASP (2024) to mitigate potential issues using the STRIDE¹ classification. Note, however, that security controls may be subject to certain changes and/or adaptations (e.g., new security solutions, deletions, modifications of realization), verification (e.g., formal, empirical), and reuse (e.g., in the same domain or across domains), while the structure of the main software architecture can be maintained. The selected security controls proposed in this example were principally identified empirically by security experts.

To ease the selection and expression of appropriate security controls and add more flexibility for software developers, mainly architects, we introduce a set of properties specifying security requirements of the modeled system. These properties restrict the operations of the system

to secure it against detected vulnerabilities (also modeled as properties). The remainder of this paper proposes an approach using formal methods and reusable libraries of vulnerabilities and requirements which have been pre-specified.

3.2. Approach overview

We propose a two-step approach to support the specification and verification of reusable security solutions for specifying software architectures as depicted in Fig. 3. The first step focuses on the modeling activities, while the second step focuses on the application itself. Both steps consider two viewpoints: software architecture and security. In the following, we provide a general overview of these steps.

3.2.1. Modeling framework development process

This step includes both modeling and formalization activities. We benefit from the advantages of modeling to abstract implementation details from the analysis and focus on supporting the vulnerability identification.

Modeling. This step consists in specifying the models using a technology-independent specification language such as first-order logic and modal logic.

The entry point of the approach is a component-port-connector metamodel, manually created with high-level concepts of the software architecture. This model is represented as an abstract system computing model (Section 4.1), enabling us to capture the basic communication primitives of the component-port-connector metamodel and the actions of both legitimate and illegitimate message providers and consumers.

¹ Microsoft STRIDE is security model which categorizes different types of threats and simplifies the overall security conversations. It classifies threats into six categories: *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service*, and *Elevation of Privilege*.

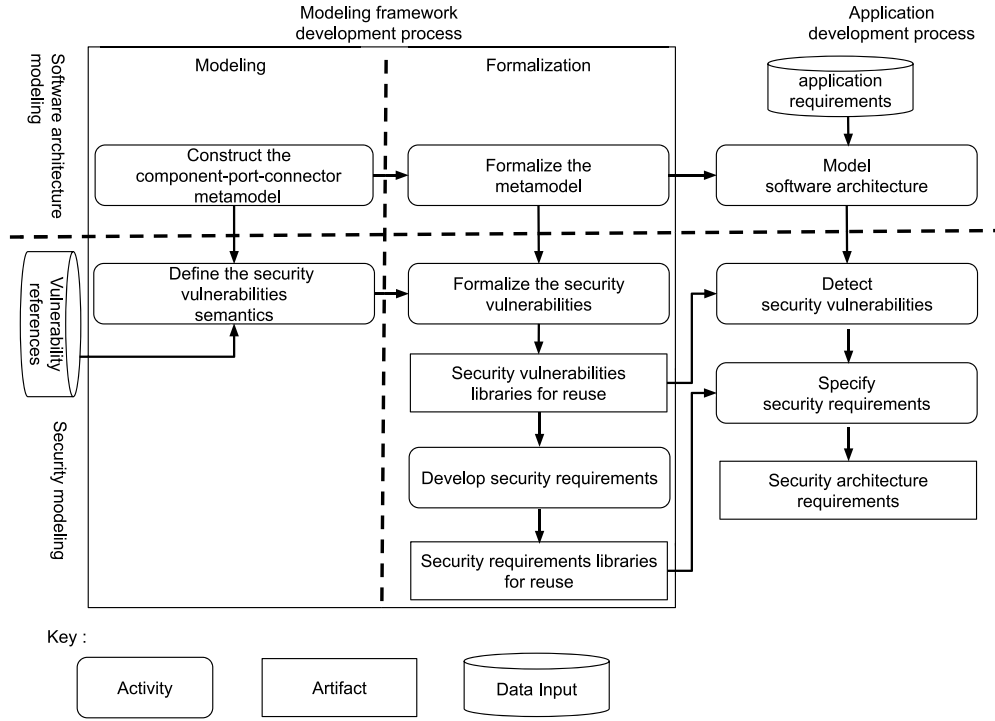


Fig. 3. Overview of the proposed approach.

Then, we model the semantics of a set of security vulnerabilities (Section 4.2) we select from reference databases such as CWE (MITRE Corporation, 2024b) or CAPEC (MITRE Corporation, 2024a). In the context of our work, the security vulnerabilities are specified in terms of a set of security flaws, representing undesirable properties—properties that should not evaluate to true for the system. Subsequently, security requirements are then introduced according to expected security properties.

Formalization. The component-port-connector metamodel and the security vulnerability semantics are formalized using a suitable formal language with tool support for model checking and verification (Section 5). By doing so, we obtain a formal architecture metamodel and a formal specification of some representative security vulnerabilities (for instance, from the CWE database). This gives us a set of reusable vulnerability libraries of properties (Section 5.2) which support the identification of security vulnerabilities in a concrete software architecture model.

Once the security vulnerabilities are identified, we propose security requirements to mitigate these vulnerabilities. The result is a set of formal security models (discussed in Section 5.2) that can be easily reused to treat the identified vulnerabilities.

This formalization step is required to leverage available tool support used in software and system modeling and to enable straightforward instantiation and model checking capabilities to support the elicitation of an appropriate set of security requirements to treat any detected vulnerabilities. As described in Section 5, in this paper we elect to use Alloy for this formalization due to its simplicity and straightforward usage of the Analyzer tool to verify properties of the formally specified system architecture models.

3.2.2. Application development process

We build a concrete architecture by instantiating the abstract architecture metamodel into a concrete model for an application-specific system. Then, we use the security vulnerability libraries to identify potential vulnerabilities in the system model by checking that it satisfies the desired properties. If the model does not satisfy the properties, we use our developed libraries to suggest new security requirements to treat the identified vulnerabilities.

4. Logical specification of vulnerabilities

In this section, we present the precise definition of a set of vulnerability categories using first-order logic and modal logic as a formalism. The main reason that drove this decision is that this formalism is abstract and technology-independent. This provides a more generic and understandable approach with mapping support to different existing property specification languages used in modeling and software development.

4.1. Abstract system computing model

A distributed software system is modeled by a set of components and a set of connectors. Components communicate and synchronize by sending and receiving messages through existing connectors. A computation program encodes the local actions that components may perform. The actions of the program include modifying local variables, sending messages, and receiving messages to/from each component in the corresponding system architecture. In the context of engineering secure systems, we define the following domain to capture the notion of both legitimate and illegitimate message providers (that can *inject* legitimate or illegitimate messages) and message consumers (that can *intercept* legitimate or illegitimate messages) of the system messages. These concepts are defined on top of the basic communication primitives (e.g., *send* and *receive*):

4.1.1. Sets

- C is the set of components
- R is the set of connectors with the following predicate:
 - $\text{has_slot}(r, c)$ indicates that $r \in R$ have an available slot for storing a message for a component $c \in C$ in its buffer, i.e., the buffer is not full and the number of messages from c in this buffer is not superior to the rate allowed
- D is the set of message payloads i.e., the part of transmitted data that is the actual intended message.
- $\mathcal{M}$ is the set of messages with the following predicates:

- $\text{has_src}(m, s)$ indicates that the declared source of $m \in \mathcal{M}$ is $s \in C$, where s may not be the true sender of m .
- $\text{has_rcv}(m, r)$ indicates that the recipient of $m \in \mathcal{M}$ is $r \in C$, where r may not be the intended receiver of m .
- $\text{has_pld}(m, d)$ indicates that $m \in \mathcal{M}$ contains a payload $d \in D$.
- $\text{buffered_by}(m, r)$ indicates that $m \in \mathcal{M}$ is in the buffer of the connector $r \in \mathcal{R}$.
- $\mathcal{A}$ is the set of actions with the following elements:
 - $\text{inject}(r, m)$ denotes that a message $m \in \mathcal{M}$ is added into the connector $r \in \mathcal{R}$.
 - $\text{intercept}(r, m)$ denotes that a message $m \in \mathcal{M}$ is taken from the connector $r \in \mathcal{R}$.
 - $\text{set_src}(m, s)$ denotes that component $c \in C$ sets the declared source $s \in C$ for message $m \in \mathcal{M}$.
 - $\text{set_rcv}(m, rcv)$ denotes that component $c \in C$ sets the intended receiver $rcv \in C$ for message $m \in \mathcal{M}$.
 - $\text{set_pld}(m, d)$ denotes that component $c \in C$ sets the payload $d \in D$ for message $m \in \mathcal{M}$.
 - $\text{get_src}(m, s)$ denotes that the declared source $s \in C$ has been taken from the message $m \in \mathcal{M}$, where s is not necessarily the true sender.
 - $\text{get_rcv}(m, rcv)$ denotes that the intended receiver $rcv \in C$ has been taken from a message $m \in \mathcal{M}$, where rcv is not necessarily the true receiver.
 - $\text{get_pld}(m, d)$ denotes that the payload $d \in D$ has been got from a message $m \in \mathcal{M}$, where d is not necessarily the true payload.

4.1.2. Modalities

- $\mathbb{H}_c(a)$ is a predicate indicating that action $a \in \mathcal{A}$ **happens** for component $c \in C$ i.e., the action a is executed by the component c at current (observed) time.
- $\mathbb{F}_c^-(a)$ is a predicate indicating that action $a \in \mathcal{A}$ occurred **antecedently** for component $c \in C$ i.e., sometimes strictly before the current (observed) time.
- $\mathbb{E}_c(a)$ is a predicate indicating that action $a \in \mathcal{A}$ is **enabled** for component $c \in C$ i.e., the component c has the possibility to execute the action a but not necessarily execute it current (observed) time.
- $\mathbb{Z}_c(a)$ is a predicate indicating that action $a \in \mathcal{A}$ is **authorized** for component $c \in C$.
- $\mathbb{Z}_{c_1}(a, c_2)$ is a predicate indicating that action $a \in \mathcal{A}$ is **authorized** for component $c_1 \in C$ and the component $c_2 \in C$ is **held accountable** for assigning this authorization.

4.1.3. Macros

- $\text{sent_by}(m, s) \equiv \exists r \in \mathcal{R} \cdot \mathbb{F}_s^-(\text{inject}(r, m)) \wedge \text{has_src}(m, s)$ is a macro expressing that s in C is the true sender of message m .
- $\text{sent_to}(m, r) \equiv \exists s \in C \cdot \text{sent_by}(m, s) \wedge \text{has_rcv}(m, r)$ is a macro expressing that r in C is the true receiver of message m .

Axioms.

- $\forall a \in \mathcal{A}, \forall c \in C \cdot \mathbb{H}_c(a) \Rightarrow \mathbb{E}_c(a)$ is an axiom that states if an action a happens for a component c , then a is enabled for c .
- $\forall m \in \mathcal{M}, \forall c_1, c_2 \in C, \forall r \in \mathcal{R} \cdot \mathbb{H}_{c_1}(\text{intercept}(r, m)) \Rightarrow \mathbb{F}_{c_2}^-(\text{inject}(r, m))$ is an axiom that states if a component c_1 intercepted a message m from a connector r , then earlier m was injected into r by a component c_2 .
- $\forall m \in \mathcal{M}, \forall c_1, c_2 \in C, \forall r \in \mathcal{R} \cdot \mathbb{H}_{c_1}(\text{inject}(r, m)) \Rightarrow \mathbb{F}_{c_1}^-(\text{set_rcv}(m, c_2))$ is an axiom that states if a component c_1 injected a message m into a connector r , then earlier c_1 set the intended receiver of m as a component c_2 .

- $\forall m \in \mathcal{M}, \forall c_1, c_2 \in C, \forall r \in \mathcal{R} \cdot \mathbb{H}_{c_1}(\text{inject}(r, m)) \Rightarrow \mathbb{F}_{c_1}^-(\text{set_src}(m, c_2))$ is an axiom that states if a component c_1 injected a message m into a connector r , then earlier c_1 set the declared source of m as a component c_2 .
- $\forall m \in \mathcal{M}, \forall c \in C, \forall d \in D, \forall r \in \mathcal{R} \cdot \mathbb{H}_{c_1}(\text{inject}(r, m)) \Rightarrow \mathbb{F}_{c_1}^-(\text{set_pld}(m, d))$ is an axiom that states if a component c injected a message m into a connector r , then earlier c set the payload of m as a payload d .
- $\forall m \in \mathcal{M}, \forall c_1, c_2 \in C, \forall r \in \mathcal{R} \cdot \mathbb{E}_{c_1}(\text{get_rcv}(m, c_2)) \Rightarrow \mathbb{F}_{c_1}^-(\text{intercept}(r, m)) \wedge \text{has_rcv}(m, c_2)$ is an axiom that states if a component c_1 is able to get the intended receiver c_2 of message m , then earlier c_1 intercepted a message m from a connector r that contained a receiver c_2 .
- $\forall m \in \mathcal{M}, \forall c_1, c_2 \in C, \forall r \in \mathcal{R} \cdot \mathbb{E}_{c_1}(\text{get_src}(m, c_2)) \Rightarrow \mathbb{F}_{c_1}^-(\text{intercept}(r, m)) \wedge \text{has_src}(m, c_2)$ is an axiom that states if a component c_1 is able to get the source c_2 of message m , then earlier c_1 intercepted a message m from a connector r that contained a source c_2 .
- $\forall m \in \mathcal{M}, \forall c \in C, \forall d \in D, \forall r \in \mathcal{R} \cdot \mathbb{E}_c(\text{get_pld}(m, d)) \Rightarrow \mathbb{F}_c^-(\text{intercept}(r, m)) \wedge \text{has_pld}(m, d)$ is an axiom that states if a component c is able to get the payload d of message m from a connector r , then earlier c intercepted a message m that contained a payload d .

4.2. Security vulnerabilities specification

Our intent is to illustrate the approach described in Section 3 using representative vulnerability categories extracted from a list of software issues catalogued in the Common Weakness Enumeration (CWE) database,² in the context of a component-port-connector architecture model and message passing communication. In each of the following sections, we specify a representative property for each vulnerability category such that the violation of the property indicates the presence of a vulnerability. This does not mean that satisfaction of the property guarantees the absence of the vulnerability, i.e., the property does not represent a sufficient condition for the vulnerability absence. In other words, we can only guarantee the presence of vulnerabilities. While satisfying these properties offer some assurance and traceability in specifying the appropriate security requirements to mitigate vulnerabilities, it does not ensure that the set of security requirements is complete.

4.2.1. Information exposure:

The CWE database defines information exposure³ as the intentional or unintentional disclosure of information to an actor that is not explicitly authorized to have access to that information. In the context of a connector-port-connector architecture model, this vulnerability occurs when components other than those for which a message was intended are able to get the payload of the sent message. Therefore, if a component c is able to get the payload d of a message m , then c must be an intended receiver of the message. Denoted by $\text{PayloadExposure}(c, m)$, this property can be specified as follows:

$$\forall d \cdot \mathbb{E}_c(\text{get_pld}(m, d)) \Rightarrow \text{sent_to}(m, c) \quad (1)$$

Coming back to the example described in Section 3.1, this vulnerability can be found when somebody is able to intercept and read the content of a request from a client (i.e., any *User*, whether malicious or not) visualizing a book page on the web server. Consequently, the invalidity of the property $\text{PayloadExposure}(\text{Webserver}, m)$ with m being any message exchanged from the *User* to *Webserver* would show that such vulnerability exists.

² <https://cwe.mitre.org>

³ <https://cwe.mitre.org/data/definitions/200.html>

4.2.2. Improper authentication:

According to the CWE database, the improper authentication⁴ vulnerability occurs when an actor claims to have a given identity, and the software does not prove or insufficiently proves that the claim is correct. In the context of a connector-port-connector architecture model, this vulnerability occurs when a component of the system cannot ensure that the sender of a message is actually the originator of the message. Therefore, if a component c_2 can get the declared source c_1 of a message m , then c_1 corresponds to the true origin of the message m . Denoted by $SenderSpoofing(c_1, c_2, m)$, this property can be specified as follows:

$$\mathbb{E}_{c_2}(\text{get_src}(m, c_1)) \Rightarrow \text{sent_by}(m, c_1) \quad (2)$$

Coming back to the illustrative example, the proposed vulnerability can be found when the web server is not able to authenticate a request from a client visualizing a book page. Consequently, the invalidity of the property $SenderSpoofing(Usr, Webserver, m)$ with m being any message exchange between the Usr and the $Webserver$ would show that such vulnerability exists.

4.2.3. Improper privilege management:

According to the CWE database, improper privilege management⁵ occurs when the software does not properly assign, modify, track, or check privileges for an actor, creating an unintended sphere of control for that actor. In the context of a connector-port-connector architecture model, this vulnerability occurs when a component is authorized to do *inject* or *intercept* operation and the system is not able to identify who assigned this privilege to it. Therefore, if a component c_1 is authorized to *inject* a message m then a component c_2 other than the component c_1 must have assigned the privilege for this action to c_1 . Similarly, if a component c_1 is authorized to *intercept* a message m then a component c_2 other than the component c_1 must have assigned the privilege for this action to c_1 . Denoted by $BadPrivilegeManagement(c_1, m)$, this property can be specified as follows:

$$\begin{aligned} &(\forall r \cdot \mathbb{Z}_{c_1}(\text{inject}(c_1, r)m) \Rightarrow \\ &\exists c_2, \text{inject} \cdot c_2 \neq c_1 \wedge \mathbb{Z}_{c_1}(\text{inject}, c_2)) \wedge \\ &(\forall r \cdot \mathbb{Z}_{c_1}(\text{intercept}(c_1, r)m) \Rightarrow \\ &\exists c_2, \text{intercept} \cdot c_2 \neq c_1 \wedge \mathbb{Z}_{c_1}(\text{intercept}, c_2)) \end{aligned} \quad (3)$$

Coming back to the example, the vulnerability can be found when an administrator is not able to track who performed the addition of an existing authorization access to the database. Consequently, the invalidity of the property $BadPrivilegeManagement(Database, m)$ with m being any message exchange with the $Database$ would show that such vulnerability exists.

4.2.4. Uncontrolled resource consumption:

According to the CWE database, uncontrolled resource consumption⁶ occurs when the software does not properly control the allocation and maintenance of a limited resource, thereby enabling an actor to influence the amount of resources consumed, eventually leading to the exhaustion of available resources. In the context of a connector-port-connector architecture model, this vulnerability occurs when a component sends a message through an already full connector or exceeds its limited rate of submission. Therefore, if a component c can inject a message m using a connector r , then the buffer of r is not full and the number of messages from c already in the buffer is inferior to the limited allowed rate. Denoted by $ConnectorBufferLimitedRate(r, m)$, this property can be specified as follows:

$$\forall c \cdot \mathbb{E}_c(\text{inject}(c, r)m) \Rightarrow \text{has_slot}(r, c) \quad (4)$$

Coming back to the illustrative example, the proposed vulnerability can be found when clients (i.e., any Usr , whether malicious or not) make more concurrent requests to visualize a book page on the web server than the capacity of the connector used for the transmissions. Consequently, the invalidity of the property $ConnectorBufferLimitedRate(c, m)$ property for any m request from the Usr to the $Webserver$ on the connector c would show that such vulnerability exists.

4.2.5. Missing release of resource after effective lifetime:

According to CWE database, the missing release of resource after effective lifetime⁷ vulnerability occurs when the software does not release a resource after its effective lifetime has ended, i.e., after the resource is no longer needed. In the context of a connector-port-connector architecture model, this vulnerability occurs when a connector keeps buffered messages that have been received by the intended receiver. Therefore, if a component c intercepted a message m from connector r , then m must not be buffered by r anymore. Denoted by $UnreleasedBuffer(r, m)$, this property can be specified as follow :

$$\forall c \cdot \mathbb{F}_c^-(\text{intercept}(r, m)) \Rightarrow \neg \text{buffered_by}(m, r) \quad (5)$$

Coming back to the example, the vulnerability can be found when a request from a client (i.e., any Usr , whether malicious or not) to visualize book page on the web server is kept in the connector used for the transmission, while the book page has already been delivered. Consequently, the invalidity of the property $UnreleasedBuffer(r, m)$ property with m being any message exchange from the Usr to the $Webserver$ on the connector r would show that such vulnerability exists.

5. Implementation in alloy

In this section, our software architecture metamodel incorporating the concepts of a component-port-connector architecture is formalized using a suitable tool formal language. It also provides the specification and verification of representative vulnerabilities from each vulnerability category defined in Section 4.

5.1. Software architecture and communication model

We propose to build a modeling framework to define architectural models that are conceptually close to the industrial practice, i.e., containing a UML-like (OMG, 2017) and UCM-like (OMG, 2022) vocabulary. Fig. 4 presents our metamodel as a class diagram. The metamodel provides concepts for describing software architectures in terms of different views (Kruchten, 1995), focusing on the:

- *Logical view* to capture the functional architecture of the system in terms of component hardware taking into account the distributed aspects;
- *Scenario view* which builds upon the logical view, describing the behavioral aspects of the architecture elements with respect to the targeted computing model;
- *Property view* for describing properties of the model to build libraries of categorized properties for reuse. These libraries are then used to type the properties of components and connectors.

⁴ <https://cwe.mitre.org/data/definitions/287.html>

⁵ <https://cwe.mitre.org/data/definitions/269.html>

⁶ <https://cwe.mitre.org/data/definitions/400.html>

⁷ <https://cwe.mitre.org/data/definitions/772.html>

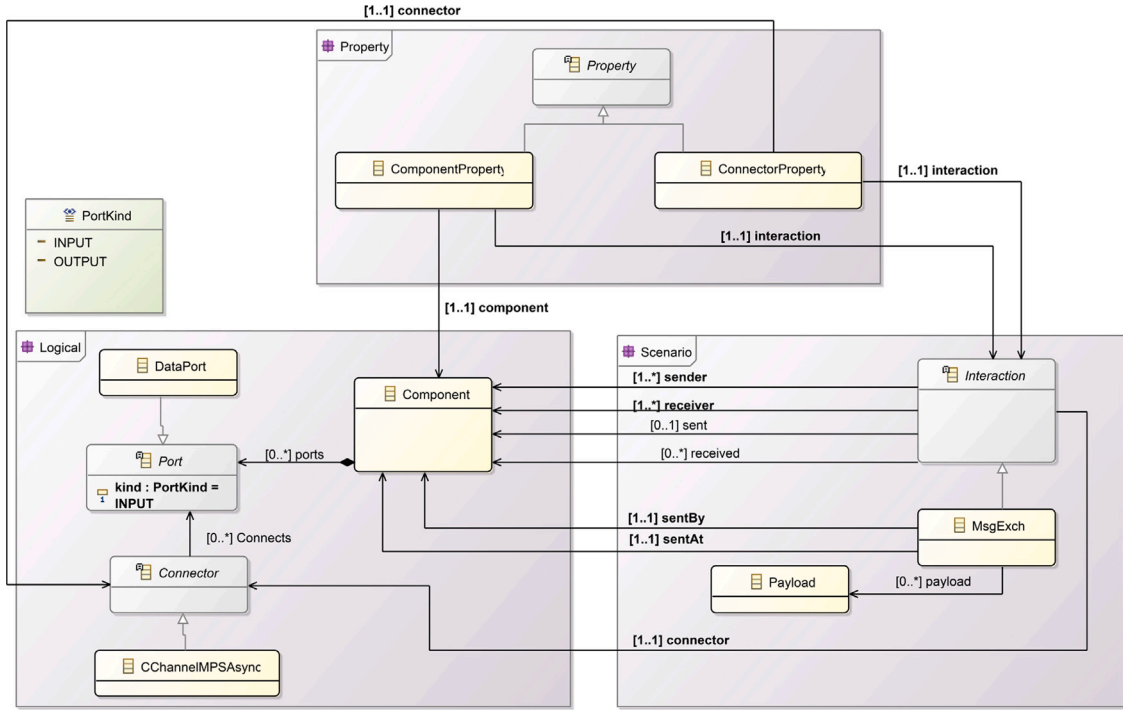


Fig. 4. Component-port-connector metamodel.

5.1.1. Logical view

A software architecture as visualized in Fig. 4 is formalized in Alloy as follows. An architectural *Component*, *Port*, *Connector*, and *DataPort* are mapped to their namesake types in Alloy and are represented as a set of Alloy signatures as depicted in Listing 1.

```

1 sig Component {
2   fun Component.ports[]: set Port { ownedBy.this }
3   fun Component.inputs[]: set Port { p:Port | p in
      ownedBy.this and p.kind = INPUT }
4   fun Component.outputs[]: set Port { p:Port | p
      in ownedBy.this and p.kind = OUTPUT }
5   abstract sig Port {
6     ownedBy: one Component,
7     kind: one PortKind
8   }
9   enum PortKind { INPUT, OUTPUT }
10  sig DataPort extends Port {}
11  abstract sig Connector {
12    connects: set Port
13  }
14  fun Connector.inputs[]: set Port { p:Port | p in
      this.connects and p.kind = OUTPUT }
15  fun Connector.outputs[]: set Port { p:Port | p
      in this.connects and p.kind = INPUT }

```

Listing 1: Software architecture metamodel in Alloy

5.1.2. Scenario view

To model the behavior of the system, we consider abstract time in the form of a parameter to express instants of occurrences of actions. An execution of a system is a sequence of steps (ticks), where a step is determined by two successive time points. To facilitate message passing communication within the connector, we present the formal specification of the connector named *CChannelMPSAsync*. This connector serves as a message buffer, and we define the appropriate communication primitives (*send* and *receive*), as illustrated in Listing 2. For further elaboration and details of the *CChannelMPSAsync* connector, please refer to Appendix A.

```

1 abstract sig CChannelMPSAsync extends Connector
2   {}
3   ...
4   abstract sig MsgExch extends Interaction {
5     payload: set Payload,
6     sentBy: one Component,
7     sentAt: one Tick
8   }
9   ...
10  abstract sig Payload {
11    fun buffered[con:Connector]: Interaction -> Tick
12    { ... }
13  }
14  pred Component.send[c:Component, m:MsgExch, t:Tick]
15  { ... }
16  pred Component.receive[c:Component, m:MsgExch, t:
17  Tick] { ... }

```

Listing 2: Message passing communication primitives

As shown in Listing 3 for the *inject* action, on top the previously defined communication primitive (i.e. *send* and *receive*) we encode the set of actions described in Section 4.1 considering the notion of execution steps in the context of an asynchronous message-passing system using the as follows:

- $\mathcal{T}$ is the sequence of execution steps
- Each action $\text{act}(\text{param}_1, \dots, \text{param}_n) \in \mathcal{A}$ is transformed to $\text{act}(\text{param}_1, \dots, \text{param}_n, t)$ denoting the execution step $t \in \mathcal{T}$ when the action occurs. For example, $\text{inject}(c, m)$ is transformed to $\text{inject}(c, m, t)$ indicating that a sender $c \in \mathcal{C}$ adds a message $m \in \mathcal{M}$ into the system at step $t \in \mathcal{T}$ and is specify using the *send* communication primitive.
- Each macro $\text{mc}(\text{param}_1, \dots, \text{param}_n)$ is transformed to $\text{mc}(\text{param}_1, \dots, \text{param}_n, t)$ denoting its relation to the execution step $t \in \mathcal{T}$. For example, $\text{sent_by}(c, m)$ is transformed. to $\text{sent_by}(c, m, t)$ indicating that a sender $c \in \mathcal{C}$ added a message $m \in \mathcal{M}$ into the system before the step $t \in \mathcal{T}$

- Each modality $\text{mod}(param_1, \dots, param_n)$ is transformed to a predicate $\text{mod}(param_1, \dots, param_n, t)$ denoting its relation to the execution step $t \in \mathcal{T}$. For example, $\mathbb{E}_c(\text{inject}(c, m))$ is transformed to $\text{E_inject}(c, m, t)$ indicating that a component $c \in C$ is able to inject a message $m \in \mathcal{M}$ into the system at the step $t \in \mathcal{T}$.

```

1  pred E_inject[c:Component, m:MsgPassing, t:Tick] {
2    some con:ConnectorMPS, p:c.uses
3    | p in con.connects.t and p.kind = OUTPUT
4    and m.payload.type = p.realizes.datatypes}
5  pred inject[c1:Component, m:MsgPassing, t:Tick] {
6    E_inject[c1, m, t]
7    c1.send[m.receiver, m.payload, t]}
8  pred injected[c:Component, m:MsgPassing, t1:Tick]
9    {
10   one t2:t1.prevs | inject[c, m, t2]}

```

Listing 3: Example for the inject action in Alloy

Additionally, we encode the set of the axioms described in Section 4.1 as facts (i.e., always true for any model instance). Listing 4 depicts the corresponding Alloy code for the fourth axiom presented stating that if a component $c1$ injected a message m into a connector r , then earlier $c1$ set the source of m as a component $c2$

```

1  fact {
2    all m:MsgPassingConstraint, c1:Component, t1:
3      Tick |
4      H_inject[c1, m, t1] => some c2:Component, t2:t1
5      .prevs | H_set_src[c1, m, c2, t2]
6  }

```

Listing 4: Axiom in Alloy

5.1.3. Property view

We encode the property view as described in Section 5.1 by defining the corresponding concepts `ComponentProperty` and `ConnectorProperty` as abstract signatures in Alloy. For simplicity, a helper function is defined to be able to verify that the property holds in a given model. As we will see in the following section, the vulnerabilities will be encoded as `ComponentProperty` or `ConnectorProperty`.

```

1  abstract sig Property {}
2  abstract sig ComponentProperty extends Property {
3    c: one Component,
4    m: one Interaction
5  }
6  pred ComponentProperty.holds[c1:one Component, msg:
7    one Interaction] {
8    { some p:this { p.c = c1 and p.m = msg }}
9  }
10 abstract sig ConnectorProperty extends Property{
11   con: one Connector,
12   m: one Interaction
13 }
14 fun ConnectorProperty.c1: one Component{ c:
15   Component | c.ports in this.con.connects }
16 fun ConnectorProperty.c2: one Component{ c:
17   Component | c.ports in this.con.connects }
18 pred ConnectorProperty.holds[conn:one Connector,
19   msg: one Interaction] {
20   { this.con = conn and this.m = msg and msg.
21     connector = conn }
22 }

```

Listing 5: Property metamodel in Alloy

5.2. Security vulnerabilities and mitigations in alloy

To formalize security vulnerabilities in Alloy models, we follow a systematic approach:

1. The vulnerability is defined as a property (either at the connector or the component level) by creating a corresponding Alloy signature.
2. Using an assertion, we check if an instance of the vulnerability can be found, i.e., if the assertion returns a counterexample corresponding to an instance of the model where the vulnerability occurs. At this step, we expect that such a counterexample will be found as no security solution is introduced yet.

Subsequently, for each identified vulnerability we propose a treatment using the following steps:

1. A security requirement is defined as a predicate (also either at the connector or the component level depending on the vulnerability) to mitigate the vulnerability.
2. We use another assertion to verify that by enforcing this security requirement in the model no instance of the vulnerability can be found anymore, i.e., no counterexample is found within the specified scope by the Alloy Analyzer.

In the following, we demonstrate this formalization process for the *PayloadExposure* vulnerability and its associated security requirement *OnlySenderAndReceiverCanConnect*. Similar processes are applied to other vulnerabilities such as *SenderSpoofing*, *BadPrivilegeVerification*, and *ConnectorBufferLimitedRate*, each with their respective security requirements. Further details regarding these vulnerabilities are provided in Appendix B.

Information exposure. The *PayloadExposure* vulnerability, defined in Listing 6, exposes the payload of a message exchanged through a connector to unauthorized components.

```

1  sig PayloadExposure extends ConnectorProperty {}{
2    not( all t:Tick | all d:Payload |
3      E_get_pld[c2, m, d, t] => sent_to[m, c2, t] )
4  }
5  assert PayloadExposureExists {
6    all con:Connector, m:MsgExch |
7    no v:PayloadExposure | v.holds[con, m]
8  }
9  check PayloadExposureExists for 5

```

Listing 6: PayloadExposure Vulnerability

Executing the *PayloadExposureExists* assertion within a scope of 5, considering any possible instances found by Alloy with 0 to 5 Connectors, 0 to 5 *MsgExch*, etc. reveals a counterexample (Fig. 5), indicating the presence of the vulnerability within the model.

To mitigate this vulnerability, we propose the *OnlySenderAndReceiverCanConnect* security requirement (Listing 7). This requirement ensures that only the sender and receiver of a message can be connected to the relevant connector, thus preventing unauthorized access to message payloads.

Executing the *OnlySenderAndReceiverCanConnectMitigatePayloadExposure* assertion confirms the effectiveness of the *OnlySenderAndReceiverCanConnect* requirement in mitigating the *PayloadExposure* vulnerability, as no counterexamples are found within the specified scope.

6. Tool support

In this section, we introduce the tool that we have developed to support the approach discussed in Section 3. The source code of the tool may be accessed at <https://gitlab.com/semcoproject/toc>.

We have implemented a prototype to facilitate the adoption of the proposed approach as an Eclipse plug-in. The fundamental structure is derived from the software architecture metamodel outlined in Fig. 4, which functions as the metamodel for a Domain-Specific Language (DSL) used in software architecture. The metamodel represents the

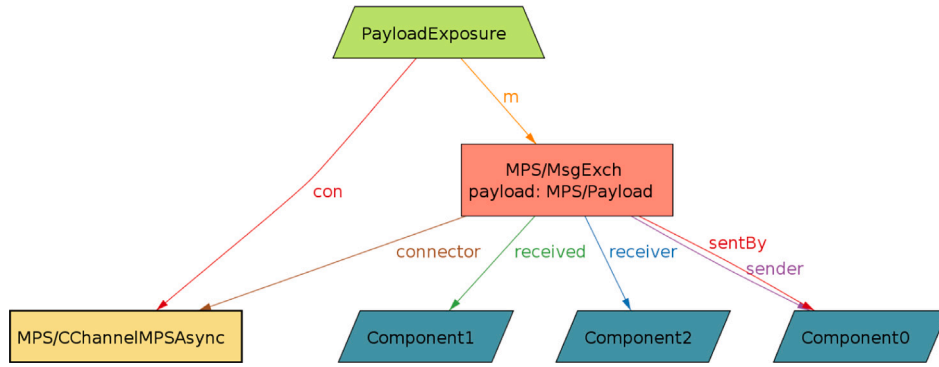


Fig. 5. Payload Exposure Counterexample.

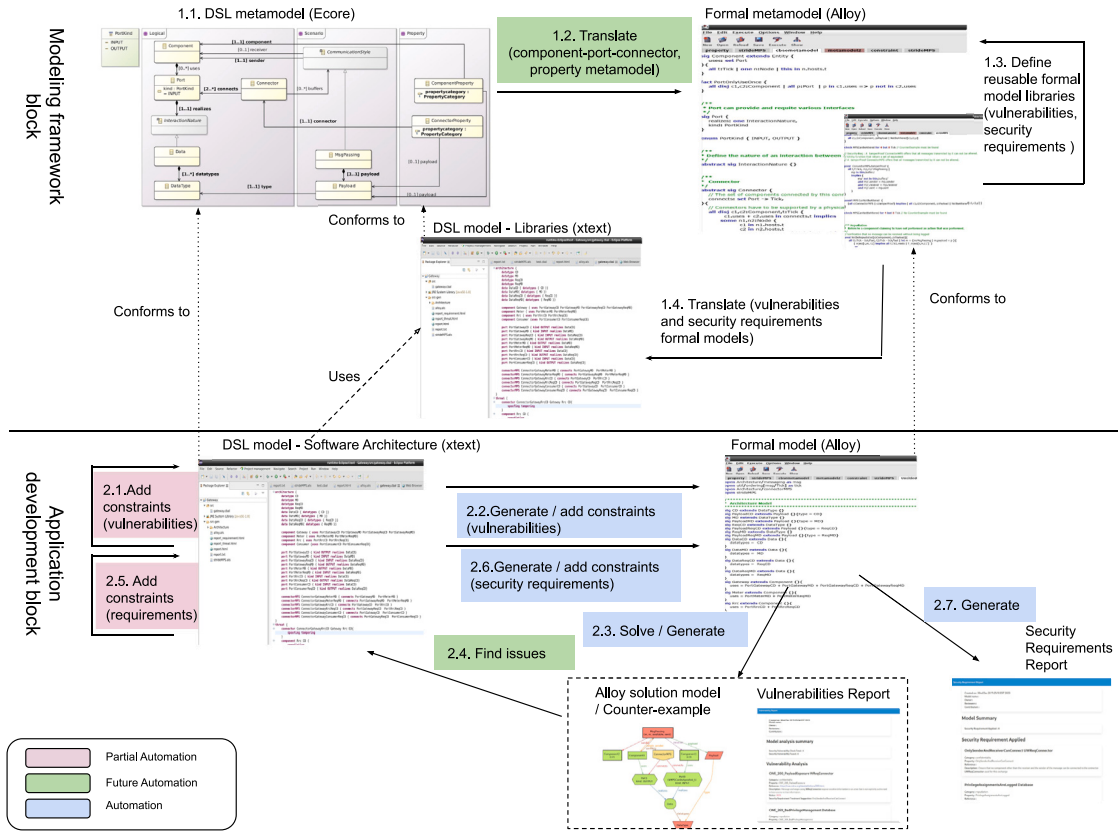


Fig. 6. Architecture and artifacts of the tool supporting the methodology.

```

1  pred Connector.OnlySenderAndReceiverCanConnect {
2    all m:MsgExch | this = m.connector
3    => this.connects = m.sender.ports + m.receiver.ports
4  }
5  assert OnlySenderAndReceiverCanConnectMitigatePayloadExposure {
6    all con:Connector | con.OnlySenderAndReceiverCanConnect
7    => all m:MsgExch | no v:PayloadExposure |
8      v.holds[con,m]
9  }
10 check OnlySenderAndReceiverCanConnectMitigatePayloadExposure for 5

```

Listing 7: PayloadExposure Vulnerability Treatment

abstract structure of the DSL, defining the elements, interfaces, and connections in the software architecture domain.

The architectural representation of the tool, as shown in Fig. 6, consists of two main components: (1) the Modeling Framework Block and (2) the Application Development Block.

6.1. Modeling framework block

The Modeling Framework Block is composed of four activities. Activity A1.1 entails the creation of the DSL metamodel (Fig. 4) using Ecore. This metamodel is then formally specified using Alloy in Activity

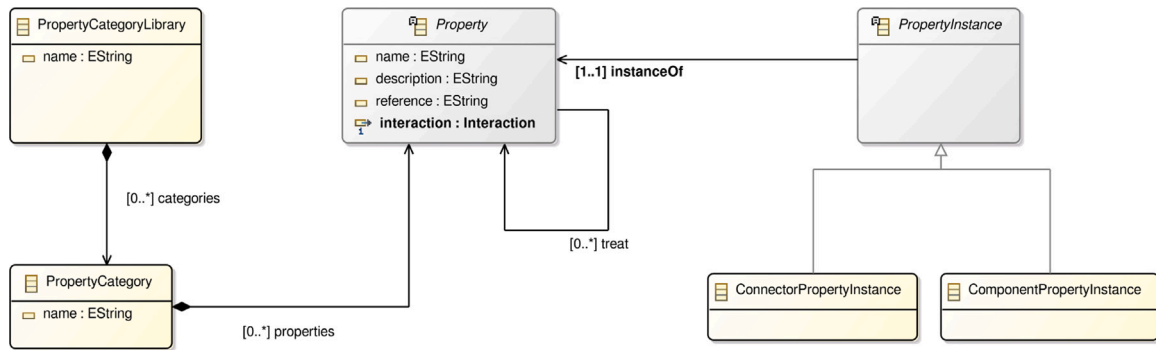


Fig. 7. Property views extensions for the DSL libraries.

```

1 library vulnerability CWE {
2   category confidentiality {
3     vulnerability CWE_200_PayloadExposure(CONNECTOR con) {
4       reference 'https://cwe.mitre.org/data/definitions/200.html'
5       comment 'Message exchanges using {{ con }} expose sensitive information to an actor that is not
6         explicitly authorized to have access to that information.'
7     }
8   }
9 }

```

Listing 8: Vulnerability DSL specification as a reusable library

```

1 library SRL {
2   category confidentiality {
3     securityRequirement OnlySenderAndReceiverCanConnect(CONNECTOR con) {
4       reference ''
5       comment 'Ensure that no component other than the receiver and the sender of the message can be connected
6         to the connector {{ con }} used for this exchange'
7       treat CWE_200_PayloadExposure
8     }
9   }
10 }

```

Listing 9: Security requirement DSL specification as a reusable library

A1.2, as described in Section 5.1. In addition, we designate the libraries for vulnerability models and security needs models as Alloy models, as outlined in Section 5.2 (A1.3). The last activity (A1.4) consists of creating a collection of DSL model libraries based on the Alloy specification of vulnerabilities and policies (formal model libraries). This is done by extending the property view of the DSL metamodel (Fig. 4) as shown in Fig. 7. This extension allows defining and instantiating the specified formal properties (i.e., vulnerabilities and security requirements in Alloy) in the DSL when designing the system (i.e., creating “headers” in the DSL for the formally defined properties in Alloy). For instance as shown in Listing 8, the definition of the property category *confidentiality* is depicted as a *PropertyCategory*. Additionally, *CWE_200_PayloadExposure* is defined as a *ConnectorProperty* within the CWE (Common Weakness Enumeration) library, and *OnlySenderAndReceiverCanConnect* is defined as a *ConnectorProperty* within the SRL (Security Requirements Library), which represents the security requirement. *Reference* and *comment* are added to provide information regarding vulnerabilities. Note that the parameters can be represented using placeholders `{{ <parameter> }}` and will be replaced by the actual value of the parameter in reports. Finally, we establish that the *OnlySenderAndReceiverCanConnect* requirement *treats* the *CWE_200_PayloadExposure* vulnerability according to the verification in Alloy (see Section 5.2). This link is used to suggest appropriate security requirements based on identified vulnerabilities.

6.2. Application development block

The purpose of the second block is to enable the execution of eight activities. Activity A2.1 enables the designer to create a software

architecture (DSL model) that conforms to with the metamodel (see Listing 11 for the college library web application). Simultaneously, it enables the incorporation of vulnerability specifications by utilizing the vulnerability model libraries that were created earlier (A1.4) (as shown in Listing 13 for the use case example).

Afterward, A2.2 entails generating the formal model (in Alloy) by utilizing a transformation engine to convert a DSL model. The Alloy Analyzer is invoked repeatedly to detect the specific vulnerability (A2.3). The resulting vulnerability report can be shown as an HTML page (A2.3) (see Fig. 8 for the college library web application). Based on the examination of the report (A2.4), activity A2.5 enables the designer to integrate the appropriate security requirements into the DSL model by utilizing the DSL model libraries and the *treat* relationship between properties. The DSL model is subsequently converted into a formal Alloy model (A2.6), which automatically integrates the formal specification of the associated security policy into the generated Alloy software architecture model based on the DSL definitions. After successfully addressing all vulnerabilities, the security requirements report is generated as an HTML page (A2.7) (see Fig. 9 for the use case example).

```

1 [% for (component in components) {%]
2 sig [%=component.name%] extends Component {}

```

Listing 10: Transformation rule for components

An EMF and Xtext-based textual editor was built to facilitate the modeling of the system architecture for these eight activities. This editor offers automated suggestions for the manual integration of the

```

1  system {
2    component User { PortDataReqUW IN , PortDataRespUW OUT };
3    component Webserver { PortDataReqWU IN , PortDataRespWU OUT , PortDataReqWD IN , PortDataRespWD OUT };
4    component Database { PortDataReqDW IN , PortDataRespDW OUT };
5
6    connector UWReqConnector : User.PortDataReqUW - Webserver.PortDataReqWD ;
7    connector UWRespConnector : Webserver.PortDataRespWU - User.PortDataRespUW;
8    connector WDReqConnector : Database.PortDataReqDW - Webserver.PortDataReqWD ;
9    connector WDRspConnector : Webserver.PortDataRespWD - Database.PortDataRespDW;
10 }

```

Listing 11: DSL model of the concrete software architecture of the college library web application

vulnerability library (A2.1). It enables auto-completion for software architecture elements, suggesting potential vulnerability categories and characteristics as the designer inputs their text. In addition, a transformation engine and verification process were created to produce the formal Alloy model (A2.2 and A2.6) and the HTML reports using Epsilon (A2.5 and A2.8). The transformation process utilizes the template functionality offered by Epsilon. It applies a series of transformation rules to each idea in a DSL model, using tree traversal. This procedure generates an Alloy formal model. Listing 10 depicts an instance of rules that can be used to convert a DSL Model into an equivalent Alloy formal model. The generation technique smoothly integrates the formal definition of the specific vulnerabilities from their DSL definition into the resulting Alloy software architecture model. The Alloy Analyzer is used to validate the resulting models and provide counterexamples. The results are analyzed and translated into several reports.

7. Application on the illustrative example

In this section, we demonstrate the application of our approach (refer to Section 3.2) in identifying and addressing vulnerabilities within component-port-connector-based architecture models. This is exemplified through an application on an illustrative example (discussed in Section 3.1).

Utilizing our tool (discussed in Section 6), we begin by instantiating the previously defined component-port-connector metamodel to specify the web application model. Subsequently, we leverage vulnerability libraries to identify potential issues within the web application model. Finally, we apply the previously established vulnerability treatments to augment the set of security requirements for the library web application example. *The use case modeling artifacts are available at <https://gitlab.com/semcoproject/toc>.*

7.1. Model the software architecture

The software architecture is defined as an instance of the proposed meta-model presented in Section 5.1. In Listing 11 we use our DSL as presented in Section 6 to model the architecture of the college library web application. Listing 12 depicts the Alloy specification of the architecture of the college library web application generated from Listing 11 by the transformation engine. We define the component, ports, and connectors as simple extensions to the concepts of our software architecture metamodel. For example, the *User* is defined as an instantiation of the *Component* type.

7.2. Vulnerabilities detection

At this step, the architect can reuse the vulnerability library defined in Section 6.1 (e.g., *SenderSpoofing*, *BadPrivilegeManagement*, etc.) to seek for vulnerabilities on the concrete architecture as depicted in Listing 13 for the illustrative example according to software architecture of the college library web application architecture and its security characteristics we identify in Section 4.2. Using the tool, the corresponding Alloy code, as shown in Listing 14, is then generated and a report on vulnerabilities of the system is generated as shown in Fig. 8.

7.3. Vulnerabilities treatment

As depicted in Listing 15, the architect revisits the set of requirements reusing the previously developed security requirement formal model libraries (see Section 5.2) associated with the identified vulnerabilities (e.g., *SenderIsAuthentic*) in order to mitigate them. To help to elicit the correct security requirement, the architecture can use the suggestion provided in vulnerability report (Fig. 8). The corresponding Alloy code, as shown in Listing 16, is then generated. By construction, the resulting concrete system model (e.g., *User*, *UWReqConnector*) is secure with respect to the identified set of vulnerabilities (e.g., *SenderIsAuthentic*, *DropInjectWhenNoSlot*). At the end, a report similar to Fig. 8 is generated where this time all vulnerabilities are mitigated and a report on the security requirements is provided as shown in Fig. 9.

```

1  fact security_requirements {
2    UWReqConnector.OnlySenderAndReceiverCanConnect
3    UWReqConnector.SenderIsAuthentic
4    Database.PrivilegeAssignmentsAreLogged
5    UWReqConnector.DropInjectWhenNoSlot
6    UWReqConnector.UnreleasedBuffer
7  }

```

Listing 16: Generated Alloy specification of security requirements of the college library web application

8. Discussion

In the following section we discuss the generalization and limitations of our approach, and how our work can be integrated into the Software Development Life Cycle (SDLC).

8.1. Generalization and limitations

We maintain the domain's vulnerability categories at an abstract level to ensure independence from any specific technology or implementation, aligning with the proposed approach. This flexibility allows formal specifications to be applicable to different implementations or architectures, provided they support message-passing communication, thereby promoting reuse.

8.1.1. Vulnerability detection and mitigation across domains

This approach of abstracting vulnerabilities to maintain a high-level specification also highlights a limitation of our work. Certain vulnerabilities, like Cross-Site Request Forgery (CSRF), depend on browser and web application operations, making them difficult to specify in our current model. Addressing this could involve refining high-level vulnerabilities into more domain-specific forms, enhancing reusability while tailoring them to specific domains.

Currently, the vulnerabilities proposed are abstracted at a high level, primarily within component-based software architecture. Despite this, the commonalities across software systems suggest our approach's applicability across different contexts. However, to maximize effectiveness, especially for vulnerabilities like CSRF, we need more refined

```

1 lone sig User extends Component {}{ ports = PortDataReqUW + PortDataRespUW }
2 lone sig Webserver extends Component {}{ ports = PortDataRespWU + PortDataRespWU + PortDataReqWD +
  PortDataRepsWD}
3 lone sig Database extends Component {}{ ports = PortDataReqDW + PortDataRespDW }
4
5 lone sig PortDataReqUW extends DataPort {}{ kind = OUTPUT }
6 lone sig PortDataReqWU extends DataPort {}{ kind = INPUT}
7 lone sig PortDataRespWU extends DataPort {}{ kind = OUTPUT }
8 lone sig PortDataRespUW extends DataPort {}{ kind = INPUT}
9 lone sig PortDataReqDW extends DataPort {}{ kind = OUTPUT }
10 lone sig PortDataReqWD extends DataPort {}{ kind = INPUT }
11 lone sig PortDataRepsWD extends DataPort {}{ kind = OUTPUT }
12 lone sig PortDataRespDW extends DataPort {}{ kind = INPUT }
13
14 lone sig UWReqConnector extends CChannelMPSAsync {}{
15   inputs = PortDataReqUW
16   outputs = PortDataReqWU
17 }
18 lone sig UWRespConnector extends CChannelMPSAsync {}{
19   inputs = PortDataRespWU
20   outputs = PortDataRespUW
21 }
22 lone sig WReqConnector extends CChannelMPSAsync {}{
23   inputs = PortDataReqDW
24   outputs = PortDataReqWD
25 }
26 lone sig WRespConnector extends CChannelMPSAsync {}{
27   inputs = PortDataRepsWD
28   outputs = PortDataRespDW
29 }

```

Listing 12: Generated Alloy model of the concrete software architecture of the college library web application

```

1 vulnerabilities {
2   CWE.confidentiality.CWE_200_PayloadExposure (WebServer);
3   CWE.confidentiality.CWE_287_SenderSpoofing (User,Webserver);
4   CWE.repudiation.CWE_269_BadPrivilegeManagement(Database);
5   CWE.denyOfService.CWE_400_ConnectorBufferLimitedRate (UWReqConnector);
6   CWE.denyOfService.CWE_772_UnreleasedBuffer(UWReqConnector);
7 }

```

Listing 13: DSL model for the detection of vulnerabilities of the college library web application

```

1 assert WebServer_PayloadExposure {
2   no c:PayloadExposure | c.holds[WebServer,MsgExch]
3 }
4 check WebServer_PayloadExposure for 5
5 assert User_Webserver_SenderSpoofing {
6   no c:SenderSpoofing | c.holds[User,Webserver,MsgExch]
7 }
8 check User_Webserver_SenderSpoofing for 5
9 assert Database_BadPrivilegeManagement {
10   no c:BadPrivilegeManagement | c.holds[Database,MsgExch]
11 }
12 check Database_BadPrivilegeManagement for 5
13 assert UWReqConnector_ConnectorBufferLimitedRate {
14   no c:ConnectorBufferLimitedRate | c.holds[UWReqConnector,MsgExch]
15 }
16 check UWReqConnector_ConnectorBufferLimitedRate for 5
17 assert UWReqConnector_UnreleasedBuffer {
18   no c:UnreleasedBuffer | c.holds[UWReqConnector,MsgExch]
19 }
20 check UWReqConnector_UnreleasedBuffer for 5

```

Listing 14: Generated Alloy assertions for the detection of vulnerabilities of the college library web application

```

1 securityRequirements {
2   SC.confidentiality.OnlySenderAndReceiverCanConnect(UWReqConnector);
3   SC.authenfication.SenderIsAuthentic(Database);
4   SC.repudiation.PrivilegeAssignmentsAreLogged(UWReqConnector);
5   SC.denyOfService.DropInjectWhenNoSlot(UWReqConnector);
6   SC.denyOfService.UnreleasedBuffer(UWReqConnector);
7 }

```

Listing 15: DSL model for the specification of security requirements of the college library web application

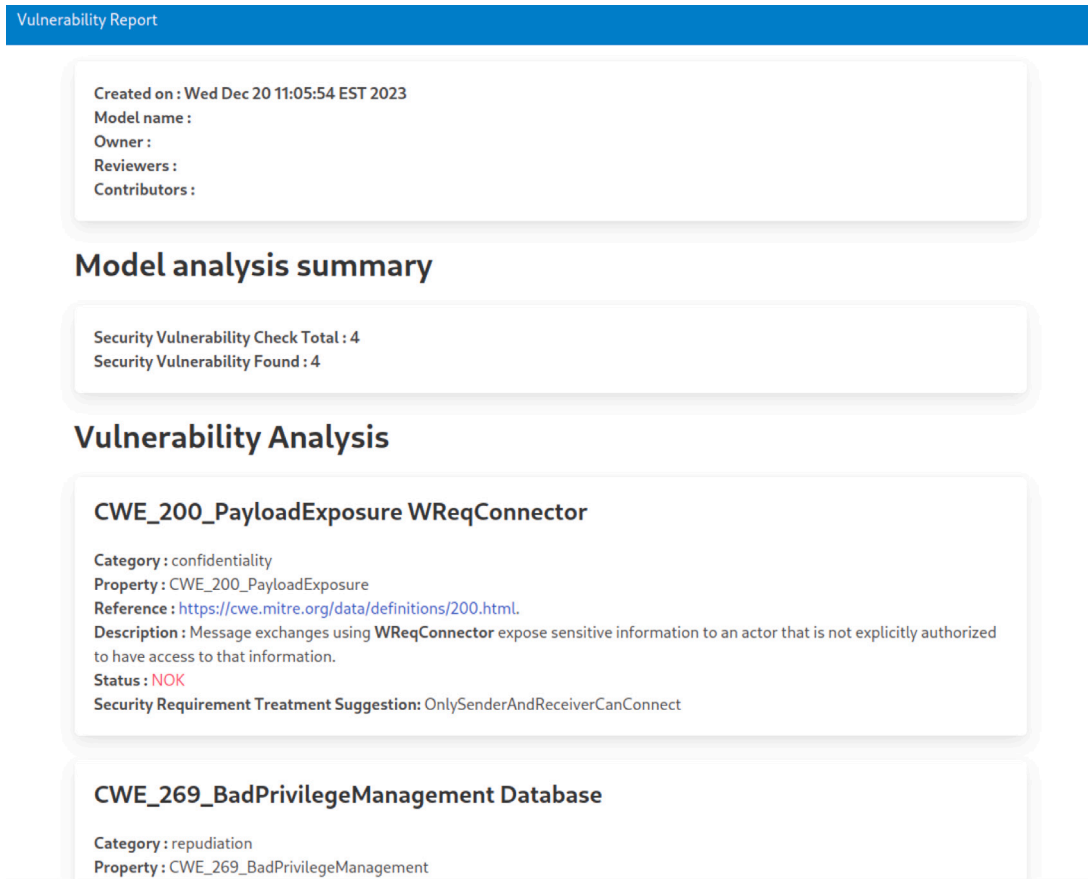


Fig. 8. Generated vulnerabilities report for of the college library web application.

Domain-Specific Languages (DSLs) tailored to different domains. These DSLs would systematically address domain-specific vulnerabilities, enhancing the precision of our methodology. A potential first step could involve integrating an ecosystem of DSLs as proposed in [Hacks and Katsikeas \(2021\)](#).

Although this paper presents a limited number of vulnerabilities, we argue that the abstraction and generality of our approach make the specified vulnerabilities representative. Our solution can accommodate a broad range of vulnerabilities, improving secure component-based software architectures. To substantiate this claim, additional research on a broader catalog of vulnerabilities, including those from CWE and other classifications, as well as exploration of various use cases, would be valuable for further validation.

8.1.2. Correctness of the vulnerability specifications

One potential limitation lies in accurately specifying CWE vulnerabilities within our formal model. We acknowledge the possibility of incomplete or incorrect specifications leading to false positives or false negatives in vulnerability detection. Nevertheless, mitigating factors can help address this issue. Since vulnerabilities are specified as reusable libraries across projects, a higher level of quality assurance can be invested in verifying the correctness of specified vulnerability libraries. Additionally, libraries could be shared, allowing them to be openly reviewed by a variety of experts across projects; for instance, an open-source repository could be made available. This approach not only improves the overall reliability of the specified library but also helps amortize the cost of verification across multiple projects.

8.1.3. Expanding the model to detect vulnerability absence

Finally, regarding the expressiveness of our formal model to detect the absence of CWE vulnerabilities, it is essential to acknowledge

that the current focus primarily lies in identifying and mitigating the presence of vulnerabilities rather than their absence. However, the absence of vulnerabilities is equally crucial for ensuring robust security measures within a system. Expanding the capabilities of our formal model to detect the absence of vulnerabilities would require a shift in perspective. We would need to incorporate mechanisms to verify not only the presence but also the absence of known vulnerabilities within the system. This would involve developing algorithms or logical frameworks to systematically analyze the system architecture and codebase to ensure that no vulnerabilities, as specified by relevant standards such as CWE, are present.

By extending the formal model to encompass both the presence and absence of vulnerabilities, we can provide a more comprehensive assessment of the system's security posture. Doing so would add enhanced assurance and traceability in specifying security requirements. This would not only ensure that known vulnerabilities are mitigated, but also the absence of known architecture-level vulnerabilities.

However, it is important to acknowledge that extending the modeling approach to include the absence of vulnerabilities may introduce additional complexity and computational overhead such leading to a state explosion during the verification process. This means that there may be trade-offs between the completeness of security requirements and practical feasibility in terms of implementation and resource constraints. Therefore, additional work needs to be performed to determine an appropriate level of model complexity and acceptable trade-offs.

8.2. The approach in systems development life cycles

The overall approach can easily fit well into various Software Development Life Cycles (SDLCs) as a supplement to the requirements specification and architecture design phases. For example, [Fig. 10](#)

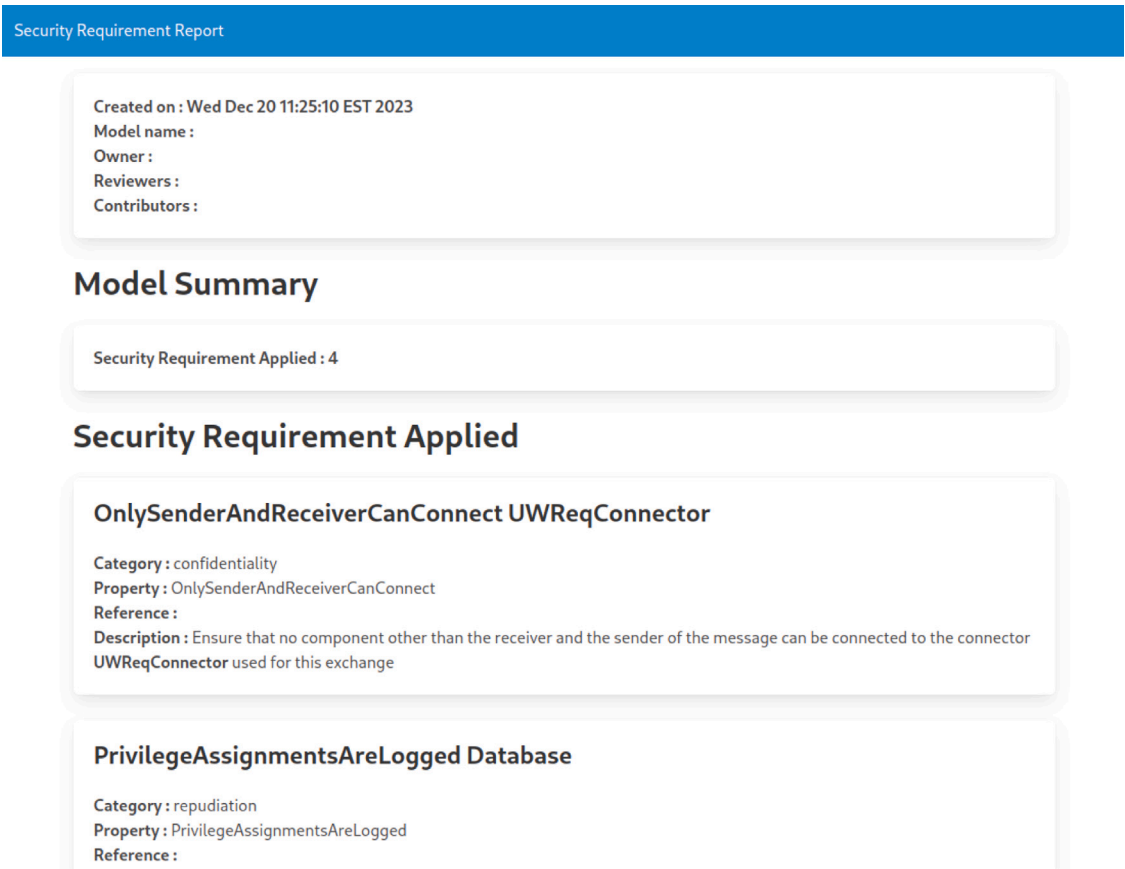


Fig. 9. Generated security requirements report for of the college library web application.

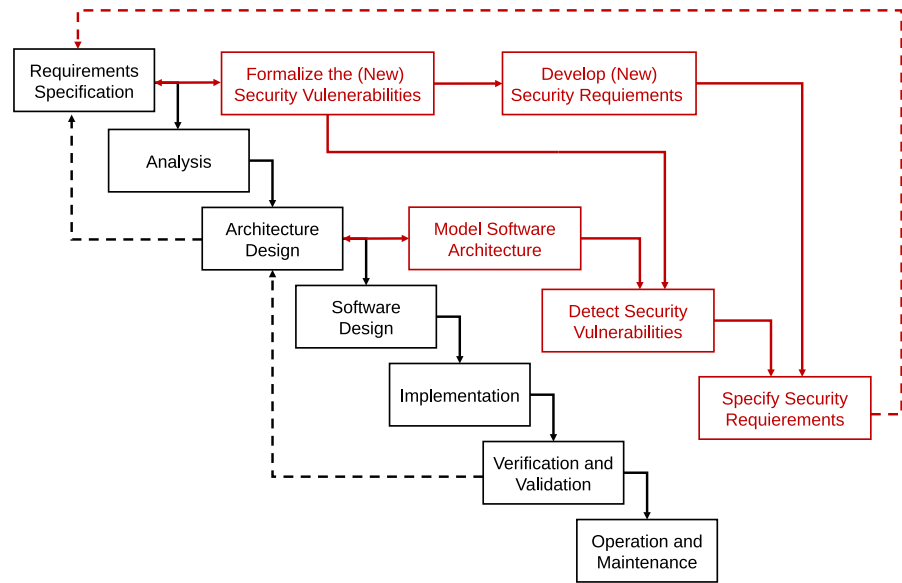


Fig. 10. The proposed approach within the Royce iterative waterfall SDLC.

shows how the approach can be integrated within the Royce iterative waterfall SDLC (Royce, 1987). As shown in Fig. 10, the activities defined in Section 3 come as a supplement to the existing phases as follows: (1) The requirements specification is extended with the “Formalize the (New) Security Vulnerabilities” and “Develop (New) Security Vulnerabilities” activities. These activities concern the modeling framework development process and should only be done if a new vulnerability or security requirement

is needed to be added in the framework, i.e., if a needed vulnerability or security requirement is not already defined in the framework libraries. (2) The architecture design is extended with the “Model Software Architecture” and “Detect Security Vulnerabilities” activities. The goal of these activities is to ensure that the software architecture model satisfies the desired properties for the designed system. (3) Finally, if we determine that the system design does not satisfy the desired properties, we take action to revisit the requirements of the system, suggesting new

security requirements in the “Specify Security Requirements” activity to protect against the detected vulnerabilities. In this way, we can iterate over the requirements specification and architecture design phases of the SDLC to revise and improve the architecture design of the system and mitigate the threats. For simplicity, we show the proposed approach within a waterfall process model, however it can similarly be integrated with other process models.

9. Related work

Kulik et al. highlight the application of formal methods to security has garnered significant attention (Kulik et al., 2022). They provide a detailed survey categorizing formal methods into theorem proving, model checking, and lightweight approaches, and discuss their use across various security-critical systems, including software architectures. It underscores the potential of formal methods to model and verify complex security properties, aligning closely with our focus on formalizing vulnerabilities in connector-port-connector architecture models.

As a result, research on vulnerabilities at the architectural level of systems has been conducted by several authors over the years (Akhawe et al., 2010; Almorsy et al., 2012, 2013; Berghe et al., 2005; Chondamrongkul et al., 2021; Elahi et al., 2010; Laibinis et al., 2016; Santos et al., 2017; Sun et al., 2011). Some authors focus on studying vulnerabilities on specific software such as Chromium (Santos et al., 2017) or on specific platforms such as the web platform (Akhawe et al., 2010), others propose a larger study with an abstract representation of vulnerabilities called *signatures* (Almorsy et al., 2012, 2013). Other authors focusing on making more approachable for designers to analyze security aspect of a design without background on formal method (Sun et al., 2011). Some approaches propose to identify attack scenarios (Elahi et al., 2010; Laibinis et al., 2016), while other authors suggest identifying vulnerabilities on the system properties rather than in the system itself (Berghe et al., 2005).

9.1. Vulnerability analysis approaches

Santos et al. proposed an empirical study of software vulnerabilities associated with architectural security tactics across Chromium, PHP, and Thunderbird, by extracting and reviewing the complete list of vulnerabilities reported for these three systems from the National Vulnerability Database (NVD)⁸ (Santos et al., 2017). While their work focuses on studying security vulnerabilities in these three projects, we provide a formalization of security vulnerabilities in the context of connector-port-connector architecture models. Besides, the approach is not tool-supported. The study is therefore empirically done, with the help of security experts and developers of the systems.

Akhawe et al. proposed a formal foundation for web security. A formal Alloy model is proposed to analyze several sample web mechanism applications and show their model can capture multiple vulnerabilities (Akhawe et al., 2010). The proposed formalization targets specific concepts of the web platform such as browsers, servers, scripts, HTTP and DNS and the way they interact, therefore compared to our approach they focus on more concrete concepts.

An Object Constraint Language (OCL) based approach is proposed by Almorsy et al. to look for vulnerabilities in system architecture on design models (Almorsy et al., 2013), and, in a lower abstraction level, in program source codes or binaries (Almorsy et al., 2012). However, the approach requires representing (in OCL) vulnerabilities in an abstract way called *signatures* (a set of invariants that indicate the presence of vulnerabilities in the analyzed program or model), which adds one preliminary step in the analysis and may complexify it. Additionally, OCL does not allow expressing constraints of dynamic

behavior that involve different states of a model at different points of time, making it difficult or impossible to specify some more complex behavior and therefore some vulnerabilities.

Another approach is proposed by Sun et al. to analyze security policies using UML/OCL as the front end to design securities policies model and the Alloy Analyzer to analyze functional aspect of security policy model (Sun et al., 2011). The idea is to remove the need for the designer to have a formal method background in order to rigorously analyze security policies. However, the approach is limited to analyzing functional behavior and focus only vulnerabilities related to access control.

Elahi et al. analyzed attack scenarios, as well as a sort of recommendations for preventing them (Elahi et al., 2010). Furthermore, the approach proposes different views to elicit security requirements, which include the requirements themselves, actors, actions, attacker templates, countermeasures, impacts of countermeasures, and a number of other information relevant to the analysis. The authors did not provide a way to identify the possible vulnerabilities on the system, highly relying on the security analysts’ experience. In this sense, this work could complement our approach, in a way that our framework could help in identifying the system vulnerabilities based on catalogs such as the CWE database, which could be used as input to their attack scenario analysis in order to help secure systems.

Gerking and Schubert presented a framework for refining and verifying information-flow security policies in cyber-physical microservice architectures, addressing the composability of secure information flows between components (Gerking and Schubert, 2019). Their approach includes a set of architectural well-formedness rules and a timing-sensitive analysis to prevent unauthorized information leaks, which is particularly relevant for real-time systems. This work focuses on the secure composition of microservices, whereas our research provides a more abstract framework for formalizing vulnerabilities in connector-port-connector architectures across diverse software designs.

9.2. Vulnerability identification techniques

Berghe et al. introduced a methodology with the purpose of categorizing potential system vulnerabilities according to their probability of emergence within a particular system (Berghe et al., 2005). This method seeks to formalize the intuitive approaches employed by security experts to anticipate security issues, linking potential vulnerabilities to the specific characteristics that delineate a system. The practical utility of this approach is demonstrated through its deployment in Web Services, where it proactively guides security measures. Conversely, our work primarily concentrates on directly identifying vulnerabilities within the architecture, rather than predicting their emergence.

The approach proposed by Laibinis et al. allows one to identify network vulnerabilities in telecommunication networks, by identifying potential attack scenarios and defining recommendations for preventing them (Laibinis et al., 2016). However, in order to identify attack scenarios, potentially harmful scenarios are *randomly* generated from the formal model simulating different service scenarios. Even though the approach helps identify ways an intruder can get access or even alter sensitive data, the fact that the process did not find an attack scenario does not prove the absence of it, given that the approach is based on a random function instead of an exhaustive analysis.

The work by Chondamrongkul et al. introduces a framework that leverages ontology and model-checking techniques for the formal security analysis of software architectures (Chondamrongkul et al., 2021). This approach is particularly noteworthy for its ability to model and analyze architectural styles such as microservices and blockchain-based systems, providing automated identification of security vulnerabilities through the combination of ontology representation and linear temporal logic assertions processed by a model checker. The framework supports dynamic tracing of security scenarios and is extensible to new architectural styles without significant reconfiguration, addressing a

⁸ <https://web.nvd.nist.gov/>

key limitation found in other formal methods approaches that often require manual and specialized knowledge to model security scenarios.

Baouya et al.'s approach aligns with our objective of using formal methods for vulnerability identification, specifically in inter-operable systems (Baouya et al., 2022). While their focus on the security challenges of interconnected systems is valuable, comparatively our framework focus on detecting architectural vulnerabilities on component based software designs — including behavior verification and reusable model libraries — offers complementary strengths.

9.3. Summary

In contrast to most methodologies relying on specific formalisms like OCL, our approach emphasizes technology independence, utilizing a language-agnostic formalism (first-order and modal logic) and a tool-supported language (Alloy), enabling broader applicability across diverse software development environments. Additionally, these methodologies are often limited either to a specific platform, domains, or restricted to verifying static systems, whereas our work proposes to detect vulnerabilities through behavior at a higher level of abstraction, and therefore presents a more general approach. Our methodology introduces formalized vulnerabilities and security requirements as reusable model libraries aimed to be used across projects, which are not addressed in most of the presented methodologies. Therefore, in this work, by leveraging the use of formal methods and model-driven techniques, our approach allows the capability to more complex vulnerabilities (behavior verification) in a reusable manner (reusable model), offering better flexibility than existing approaches.

10. Concluding remarks and future work

This paper proposes an approach to vulnerability specification, detection, and treatment of component-based software architecture models following two levels of abstraction: (1) logical specification of vulnerabilities using first-order and modal logic as an abstract and technology-independent formalism; (2) formalization and verification using Alloy as a tool language for modeling and analysis of component-based software systems. For each representative CWE vulnerability, we define and verify an appropriate property representing the security requirement that treats the vulnerability. The proposed approach can help system designers to rework their designs to mitigate the identified vulnerabilities and/or to aid in selecting appropriate security controls. In this way, our approach can aid in developing reliable software systems.

We have developed a Model-Driven Engineering (MDE) tool utilizing the Eclipse Modeling Framework to support the proposed approach. This tool encompasses a model editor that automatically translates into Alloy for the verification step, thereby obviating the necessity for architects using the framework to be proficient in the Alloy language. This contributes to the improved reusability of the developed vulnerability, extending its accessibility to a broader audience of practitioners.

As a future work, we plan to improve the approach by including the specification and classification of security patterns such as those used in Schumacher (2003). Additionally, we intend to add more formality to ensure semantic validation, by transforming existing Pattern Based System Engineering (PBSE) concepts (Fernandez, 2013; Hamid and Weber, 2018) to Alloy specifications. We will seek new opportunities to apply the proposed approach to other security classifications/models. To improve the scalability of the approach to larger-scale systems, we plan to investigate the integration of methods for decomposing the architectural model, as proposed in Baak et al. (2023). Finally, we plan to investigate the relationships between the identified vulnerabilities and other pertinent security concepts, such as threats (Rouland et al., 2021) and security objectives (Rouland et al., 2019). By understanding these relationships, we aim to develop more complementary, comprehensive, and holistic approaches to develop secure software architecture by design.

CRedit authorship contribution statement

Quentin Rouland: Writing – original draft, Validation, Software, Methodology, Conceptualization. **Brahim Hamid:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Jason Jaskolka:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Code is made available at <https://gitlab.com/semcoproject/toc>.

Acknowledgments

We like to express our gratitude to the anonymous reviewers for their valuable feedback, which has significantly enhanced the quality of this article.

Appendix A. CChannelMPASync — Alloy

To model the behavior of the system, we consider abstract time in the form of a parameter to express instants of occurrences of actions. An execution of a system is a sequence of steps (instants), where a step is determined by two successive time points. We used the *Ordering* module provided by Alloy to express a time in a discrete sense, referred to as *Tick*, where time is explicitly modeled as a set of discrete, ordered *Tick* instances. This added notion of time enables the execution to be simulated through the evolution of the status of the structural elements of the component-port-connector metamodel.

To facilitate message passing communication within the connector, we present the formal specification of the connector named *CChannelMPASync*. This connector serves as a message buffer, and we define the appropriate communication primitives, as illustrated in Listing 17.

The *CChannelMPASync* connector possesses the following key properties: (1) its definition aligns with asynchronous communication principles (Async), (2) it is a channel, i.e., establishing a connection between two components, (3) uses Message Passing Communication (MPS) support, and (4) it is size-limited; its message buffer has a finite capacity.

```

1  abstract sig Interaction {
2    connector: one Connector,
3    sender: set Component,
4    receiver: set Component,
5    received: set Component -> Tick,
6    sent: set Component -> Tick
7  }{
8    all c:Component, t:Tick | (c in sent.t implies
9      some p:Port | p in connector.inputs and p
10     in c.outputs)
11    all c:Component, t:Tick | (c in received.t
12      implies some p:Port | p in connector.
13      outputs and p in c.inputs)
14  }
15  sig CAsync in Connector {}{}
16  sig CChannel in Connector {}{
17    lone this.inputs
18    lone this.outputs
19  }
20  sig CMPS in Connector {}{
21    all i:Interaction | this = i.connector => i in
22      MsgExch // CMPS is an MsgExch Interaction
23    all p:Port | p in connects => p in DataPort //
24      MsgExch use data port
25  }
26  sig CSizeLimited in Connector {
27    size : one Int
28  }

```



```

23 abstract sig CChannelMPSAsync extends Connector
24   {}
25 fact {
26   CChannelMPSAsync in CAsync
27   CChannelMPSAsync in CChannel
28   CChannelMPSAsync in CMPS
29   CChannelMPSAsync in CSizeLimited
30 }
31 abstract sig MsgExch extends Interaction {
32   payload: set Payload,
33   sentBy: one Component,
34   sentAt: one Tick
35 }{
36   # sender = 1
37   # receiver = 1
38   all c:Component, t:Tick | (c in sent.t => (c =
39     sentBy and t = sentAt))
40   all c:Component, t:Tick | (c in received.t
41     implies one c2:Component, t2:t.prevs+t | c2
42     in sent.t2) // Received was sent
43   all c:Component, t:Tick | (c in received.t
44     implies all t2:t.nexts | c not in received.
45     t2) // AtmostOneRecvInstant
46   connector in CMPS
47 }
48 abstract sig Payload {}
49 fun buffered[con:Connector]: Interaction -> Tick
50   {{i: Interaction, t: Tick | t.gte[i.sentAt]
51     and some c:Component, t2:t.nexts + t | c in i
52     .received.t2 and i.connector = con }}
53
54 pred Component.send[c:Component, m:MsgExch, t:Tick]
55   {
56     m.receiver = c
57     this = m.sentBy
58     t = m.sentAt
59   }
60
61 pred Component.receive[c:Component, m:MsgExch, t:
62   Tick] {
63   m.sender = c
64   this = m.r
65 }

```

Listing 17: Message passing communication primitives

Appendix B. Security vulnerabilities — Alloy

B.1. Improper authentication

The *SenderSpoofing* vulnerability is defined in Listing 18 as a property within a connector according to the logical specification in Section 4.2.2.

```

1 sig SenderSpoofing extends ConnectorProperty {}{
2   not(all t:Tick | E_get_src[c2,m,c1,t] =>
3     sent_by[m,c1,t])
4 }
5 assert SenderSpoofingExists {
6   all c:Connector, m:MsgExch | no v:
7     SenderSpoofing |
8     v.holds[c,m]
9 }
10 check SenderSpoofingExists for 5

```

Listing 18: SenderSpoofing Vulnerability

We run the *SenderSpoofingExists* assertion to see if an instance of the *SenderSpoofing* vulnerability exists. The Alloy Analyzer finding a counterexample, meaning that a *SenderSpoofing* vulnerability is detected in the model. To treat the vulnerability, we suggest a security requirement on a connector called *SenderIsAuthentic* as shown in Listing 19. This ensures that any component that sends a message through this connector is the declared sender of the message.

```

1 pred Connector.SenderIsAuthentic {
2   all c:Component, m:MsgExch, t:Tick | inject[c,
3     this,m,t] => m.sender = c
4 }

```

```

4 assert SenderIsAuthenticMitigateSenderSpoofing {
5   all c:Connector | c.SenderIsAuthentic => all
6     m:MsgExch | no v:SenderSpoofing | v.
7     holds[c,m]
8 }
9 check SenderIsAuthenticMitigateSenderSpoofing for
10   5

```

Listing 19: SenderSpoofing Vulnerability Treatment

Finally, we check that the satisfaction of the *SenderIsAuthentic* property allows the treatment of the *SenderSpoofing* vulnerability by running the *SenderIsAuthenticMitigateSenderSpoofing* assertion. The Alloy Analyzer shows that no counterexample is found within the specified scope. This means that no *SenderSpoofing* vulnerability is detected in the model anymore.

B.2. Improper privilege management

The *BadPrivilegeVerification* vulnerability is defined in Listing 20 as a property within a component according to the logical specification in Section 4.2.3.

```

1 sig BadPrivilegeManagement extends
2   ComponentProperty {}{
3   not(all t:Tick |
4     Z_inject[c,m,t] => some c2:Component - c |
5     Z_inject[c, c2,m,t]
6     and
7     Z_intercept[c,m,t] => some c2:Component - c
8     |
9     Z_intercept[c, c2,m,t] )
10 }
11 assert BadPrivilegeManagementExists {
12   all c:Component, m:MsgExch | no v:
13     BadPrivilegeManagement | v.holds[c,m]
14 }
15 check BadPrivilegeManagementExists for 5

```

Listing 20: BadPrivilegeManagement Vulnerability

We run the *BadPrivilegeManagementExists* assertion to see if an instance of the *BadPrivilegeManagement* vulnerability can exist. The Alloy Analyzer shows that is possible by finding a counterexample. The existence of the counterexample means that a *BadPrivilegeManagement* vulnerability is detected in the model. Then, to treat this vulnerability, we suggest a security requirement on a component called *PrivilegeAssignmentsAreLogged* as shown in Listing 21. It ensures that registered privilege must have a corresponding entry log in the system.

Finally, we check that the satisfaction of the *PrivilegeAssignmentsAreLogged* property allows the treatment of the *BadPrivilegeManagement* vulnerability by running the *PrivilegeAssignmentsAreLoggedMitigateBadPrivilegeManagement* assertion. The Alloy Analyzer shows that no counterexample is found within the specified scope, meaning that no *BadPrivilegeManagement* vulnerability is detected in the model anymore.

B.3. Uncontrolled resource consumption

The *ConnectorBufferLimitedRate* vulnerability is defined in Listing 22 as a property within a component according to the logical specification in Section 4.2.4.

We run the *ConnectorBufferLimitedRateExists* assertion to see if an instance of the *ConnectorBufferLimitedRate* vulnerability exists. The Alloy Analyzer finds a counterexample, meaning that a *ConnectorBufferLimitedRate* vulnerability is detected in the model. To treat this vulnerability, we suggest a security requirement on a connector called *DropInjectWhenNoSlot* as shown in Listing 23. This requirement ensures that no injection of messages is accepted when the connector buffer is already full.


```

1  pred Component.PrivilegeAssignmentsAreLogged {
2    all ac: Authorization | ac.clnt = this
3    => some c:Component - this | c = ac.addBy
4  }
5
6  assert PrivilegeAssignmentsAreLoggedMitigateBadPrivilegeManagement {
7    all c:Component | c.PrivilegeAssignmentsAreLogged => all m:MsgExch | no v:BadPrivilegeManagement | v.holds
8    [c,m]
9  }
10 check PrivilegeAssignmentsAreLoggedMitigateBadPrivilegeManagement for 5

```

Listing 21: BadPrivilegeManagement Vuln. Treatment

```

1  sig ConnectorBufferLimitedRate extends ConnectorProperty {}{
2    not( all t:Tick | inject[c2,con,m,t] => has_slot[con,t])
3  }
4  assert ConnectorBufferLimitedRateExists {
5    all con:Connector, m:MsgExch | no v:ConnectorBufferLimitedRate | v.holds[con,m]
6  }
7  check ConnectorBufferLimitedRateExists for 5

```

Listing 22: ConnectorBufferLimitedRate Vulnerability

```

1  pred Connector.DropInjectWhenNoSlot {
2    all t:Tick, c:Component, m:MsgExch | E_inject[c,this,m,t] => # buffered[this].t < this.size
3  }
4  assert DropInjectWhenNoSlotMitigateConnectorBufferOverflow {
5    all con:Connector | con.DropInjectWhenNoSlot
6    => all m:MsgExch | no v:ConnectorBufferLimitedRate |
7    v.holds[con,m]
8  }
9  check DropInjectWhenNoSlotMitigateConnectorBuffer
10 Overflow for 5

```

Listing 23: ConnectorBufferLimitedRate Vuln. Treatment

Finally, we check that the satisfaction of the *DropInjectWhenNoSlot* property allows the treatment of the *ConnectorBufferLimitedRate* vulnerability by running the *DropInjectWhenNoSlotMitigateConnectorBufferLimitedRate* assertion. The Alloy Analyzer shows that no counterexample is found within the specified scope. This means that no *ConnectorBufferLimitedRate* vulnerability is detected in the model anymore.

B.3.1. Missing release of resource after effective lifetime

The *UnreleasedBuffer* vulnerability is defined in Listing 24 as a property within a component according to the logical specification in Section 4.2.5.

```

1  sig UnreleasedBuffer extends ConnectorProperty
2  {}{
3    not (all t:Tick | intercept[c1,con,m,t] =>
4    clear_buffer[con,m,t])
5  }
6  assert UnreleasedBufferExists {
7    all con:Connector, m:MsgExch | no v:
8    UnreleasedBuffer | v.holds[con,m]
9  }
10 check UnreleasedBufferExists for 5

```

Listing 24: UnreleasedBuffer Vulnerability

We run the *UnreleasedBufferExists* assertion to see if an instance of the *UnreleasedBuffer* vulnerability can exist. The Alloy Analyzer shows finds a counterexample, meaning that a *UnreleasedBuffer* vulnerability is detected in the model. Then, to treat this vulnerability, we suggest a security requirement on a connector called *NotKeepingMsg* as shown in Listing 25. The requirement ensures that when a component intercepts a message from the connector, this message is deleted from the connector buffer instantly.

```

1  pred Connector.NotKeepingMsg {
2    all c:Component, m:MsgExch, t:Tick | intercept
3    [c,this,m,t] => m not in buffered[this].t

```

```

3  }
4  assert NotKeepingMsgMitigateUnreleasedBuffer {
5    all con:Connector | con.NotKeepingMsg => all
6    m:MsgExch | no v:UnreleasedBuffer | v.
7    holds[con,m]
8  }
9  check NotKeepingMsgMitigateUnreleasedBuffer for
10 5

```

Listing 25: UnreleasedBuffer Vulnerability Treatment

Finally, we check that the satisfaction of the *NotKeepingMsg* property allows the treatment of the *UnreleasedBuffer* vulnerability by running the *NotKeepingMsgMitigateUnreleasedBuffer* assertion. The Alloy Analyzer shows that no counterexample is found within the specified scope, meaning that no *UnreleasedBuffer* vulnerability is detected in the model anymore.

References

- Akhawe, D., Barth, A., Lam, P.E., Mitchell, J., Song, D., 2010. Towards a formal foundation of web security. In: 2010 23rd IEEE Computer Security Foundations Symposium. pp. 290–304.
- Almorsy, Mohamed, Grundy, John, Ibrahim, Amani S., 2012. Supporting automated vulnerability analysis using formalized vulnerability signatures. In: Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering. In: ASE 2012, Association for Computing Machinery, New York, NY, USA, pp. 100–109.
- Almorsy, Mohamed, Grundy, John, Ibrahim, Amani S., 2013. Automated software architecture security risk analysis using formalized signatures. In: Proceedings of the 2013 International Conference on Software Engineering. ICSE '13, IEEE Press, pp. 662–671.
- Anderson, Ross, 2020. Security Engineering: A Guide to Building Dependable Distributed Systems, 3rd Edition Wiley.
- Baak, James, Rouland, Quentin, Jaskolka, Jason, 2023. A formal metamodel for software architectures with composite components. In: 12th International Conference on Model and Data Engineering. MEDI, Springer.

- Baouya, Abdelhakim, Ouchani, Samir, Bensalem, Saddek, 2022. Formal modelling and security analysis of inter-operable systems. In: *International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems*. Springer, pp. 555–567.
- Barnabe, D., Goodnight, J., Hamilton, D., Bayuk, J.L., 2011. Systems security engineering. *IEEE Secur. Priv. Mag.* 9, 72–74.
- Berghe, Chris Vanden, Riordan, James, Piessens, Frank, et al., 2005. A vulnerability taxonomy methodology applied to web services. In: *Proceedings of the 10th Nordic Workshop on Secure IT Systems. NORDSEC2005*, pp. 49–62.
- Chondamrongkul, Nacha, Sun, Jing, Warren, Ian, 2021. Formal security analysis for software architecture design: An expressive framework to emerging architectural styles. *Sci. Comput. Program.* 206, 102631.
- Cimatti, A., Clarke, E.M., Giunchiglia, F., Roveri, M., 1999. NuSMV: A new symbolic model verifier. In: Halbwachs, N., Peled, D. (Eds.), *Proceedings 11th Conference on Computer-Aided Verification*. In: *Lecture Notes in Computer Science*, (1633), Springer, pp. 495–499.
- Clarke, E.M., Emerson, E.A., 1982. Design and synthesis of synchronization skeletons using branching time temporal logic. In: Kozen, D. (Ed.), *Logics of Programs*. In: *Lecture Notes in Computer Science*, vol. 131, Springer Berlin/Heidelberg, pp. 52–71.
- Crnkovic, I., 2005. Component-based software engineering for embedded systems. In: *Proceedings of the 27th International Conference on Software Engineering. ICSE '05*, ACM, pp. 712–713.
- Elahi, Golnaz, Yu, Eric, Zannone, Nicola, 2010. A vulnerability-centric requirements engineering framework: Analyzing security attacks, countermeasures, and requirements based on vulnerabilities. *Requir. Eng.* 15 (1), 41–62.
- Fernandez, E.B., 2013. *Security patterns in practice: Building secure architectures using software patterns*. In: *Software Design Patterns*, Wiley, ISBN: 978-1-119-99894-5.
- Gerking, Christopher, Schubert, David, 2019. Component-based refinement and verification of information-flow security policies for cyber-physical microservice architectures. In: *2019 IEEE International Conference on Software Architecture. ICSEA, IEEE*, pp. 61–70.
- Hacks, Simon, Katsikeas, Sotirios, 2021. Towards an ecosystem of domain specific languages for threat modeling. In: *International Conference on Advanced Information Systems Engineering*. Springer, pp. 3–18.
- Hamid, Brahim, Weber, Donatus, 2018. Engineering secure systems: Models, patterns and empirical validation. *Comput. Secur.* 77, 315–348.
- Holzmann, Gerard, 2003. *The Spin Model Checker: Primer and Reference Manual*, First Addison-Wesley Professional.
- Jackson, D., 2006. *Software Abstractions: Logic, Language, and Analysis*. The MIT Press.
- Jackson, D., 2019. Alloy: A language and tool for exploring software designs. *Commun. ACM* 62 (9), 66–76.
- Kruchten, P., 1995. Architectural blueprints - the “4+ 1” view model of software architecture. *IEEE Softw.* 12 (6), 42–50.
- Kulik, Tomas, Dongol, Brijesh, Larsen, Peter Gorm, Macedo, Hugo Daniel, Schneider, Steve, Tran-Jørgensen, Peter WV, Woodcock, James, 2022. A survey of practical formal methods for security. *Form. Asp. Comput.* 34 (1), 1–39.
- Laibinis, Linas, Troubitsyna, Elena, Pereverzeva, Inna, Oliver, Ian, Holtmanns, Silke, 2016. A formal approach to identifying security vulnerabilities in telecommunication networks. In: *Formal Methods and Software Engineering - 18th International Conference on Formal Engineering Methods, ICFEM 2016, Tokyo, Japan, November 14–18, 2016, Proceedings*. pp. 141–158.
- Larsen, Kim G., Pettersson, Paul, Yi, Wang, 1997. UPPAAL in a nutshell. *Int. J. Softw. Tools Technol. Transfer* 1 (1–2), 134–152.
- Microsoft, 2024. Microsoft security development lifecycle (SDL). <https://www.microsoft.com/en-us/securityengineering/sdl/>. (Accessed May-2024).
- MITRE Corporation, 2024a. Common attack pattern enumeration and classification (CAPEC). <http://capec.mitre.org/>. (Accessed April-2024).
- MITRE Corporation, 2024b. Common weakness enumeration (CWE). <https://cwe.mitre.org/>. (Accessed April-2024).
- OMG, 2017. Unified modeling language (UML), version 2.5.1. <https://www.omg.org/spec/UML/>. (Accessed May-2024).
- OMG, 2022. Unified component model for distributed, real-time and embedded systems, version 1.3. <https://www.omg.org/spec/UCM/>. (Accessed May-2024).
- OWASP, 2024. Application threat modeling. https://owasp.org/www-community/Threat_Modeling_Process. (Accessed April-2024).
- Pnueli, A., 1977. The temporal logic of programs. In: *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*. pp. 46–57.
- Rouland, Quentin, Hamid, Brahim, Bodeveix, Jean-Paul, Filali, Mamoun, 2019. A formal methods approach to security requirements specification and verification. In: *2019 24th International Conference on Engineering of Complex Computer Systems. ICECCS, IEEE*, pp. 236–241.
- Rouland, Quentin, Hamid, Brahim, Jaskolka, Jason, 2021. Specification, detection, and treatment of STRIDE threats for software components: Modeling, formal methods, and tool support. *J. Syst. Archit.* 117, 102073.
- Royce, Winston W., 1987. Managing the development of large software systems: concepts and techniques. In: *Proceedings of the 9th International Conference on Software Engineering*. IEEE Computer Society Press, pp. 328–338.
- Santos, J.C.S., Peruma, A., Mirakhorli, M., Galstery, M., Vidal, J.V., Sejia, A., 2017. Understanding software vulnerabilities related to architectural security tactics: An empirical investigation of chromium, PHP and thunderbird. In: *2017 IEEE International Conference on Software Architecture. ICSEA*, pp. 69–78.
- Schumacher, M., 2003. Security engineering with patterns - origins, theoretical models, and new applications. In: *Lecture Notes in Computer Science*, vol. 2754, Springer, ISBN: 978-3-540-45180-8.
- Software Design Group (MIT), 2024. Alloy analyzer. <http://alloytools.org/>. (Accessed April-2024).
- Spivey, J.M., 1989. *The Z Notation*. Prentice-Hall.
- Sun, W., France, R., Ray, I., 2011. Rigorous analysis of UML access control policy models. In: *2011 IEEE International Symposium on Policies for Distributed Systems and Networks*. pp. 9–16.



Dr. Quentin Rouland is currently a Postdoctoral Researcher at the Université du Québec en Outaouais (Canada) in the LRSI Research Lab. Previously, he was a Postdoctoral Researcher at Carleton University (Canada) in the CyberSEA Research Lab. He earned his Ph.D. in Computer Science from the University of Toulouse Paul Sabatier (France) in 2021 as a member of the IRT-ARGOS research team. He obtained his M.Sc. in Computer Science from the University of Toulouse Jean-Jaurès (France) in 2017. His primary research interests lie in software architectures, formal methods, model-driven engineering, and security. Dr. Rouland is particularly interested in modeling and formal analysis approaches to secure software architecture.



Dr. Brahim Hamid is a professor of computer science at the University of Toulouse Jean-Jaurès and he is a member of the IRT-ARGOS team. He got his Ph.D. degree in 2007 in the area of dependability in distributed computing systems from the University of Bordeaux (France). In addition, he has an M.Sc. in Theoretical Computer Science. He has been an assistant professor (ATER) at ENSEIRB (Bordeaux, France), and a member of LaBRI (France). Then he worked as a post-doc in the modeling group at the CEA-Saclay List (France). His main research topics are software languages engineering, at both the foundations and application level, particularly for resource constrained systems. He works on security, dependability, software architectures, formalization, validation and verification. Furthermore, he is an expert in model-driven development approaches both in research and teaching. Emphasis of his work lies on the development of tools to model and to analyze secure and dependable software architecture of critical infrastructures, cyber-physical and distributed systems.



Dr. Jason Jaskolka is an Associate Professor in the Department of Systems and Computer Engineering and the Director of the Cyber Security Evaluation and Assurance (CyberSEA) Research Lab at Carleton University, Ottawa, ON, Canada. He received his Ph.D. in Software Engineering in 2015 from McMaster University, Hamilton, ON, Canada. His research interests include cyber security evaluation and assurance, threat modeling, security-by-design, and formal methods and data-driven approaches for software and security engineering. He is interested in applying his research to critical infrastructures, industrial control systems, cyber-physical and distributed systems, and the Internet of Things (IoT).